

Machine Programming

Lecture 12 – Pre-training of Coding Language Models

Ziyang Li

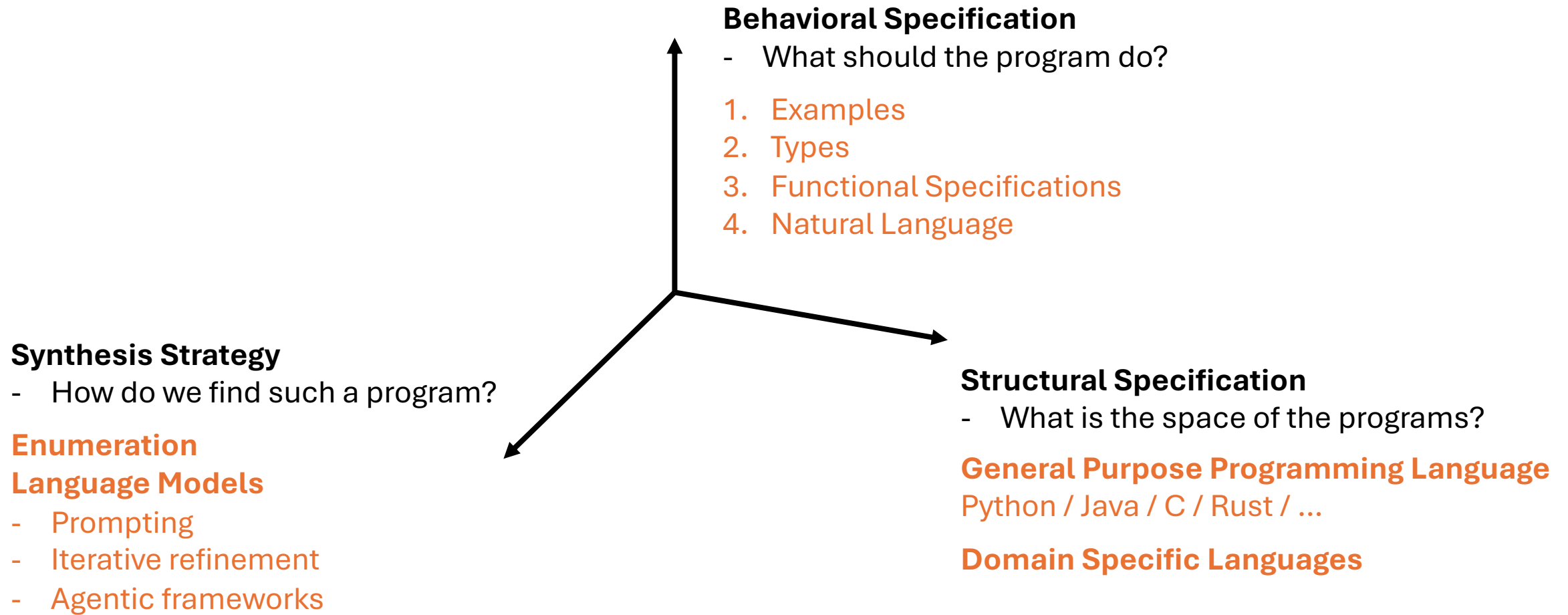


Open Source Summit North America

Minneapolis, Minnesota

May 20, 2026

The Course So Far

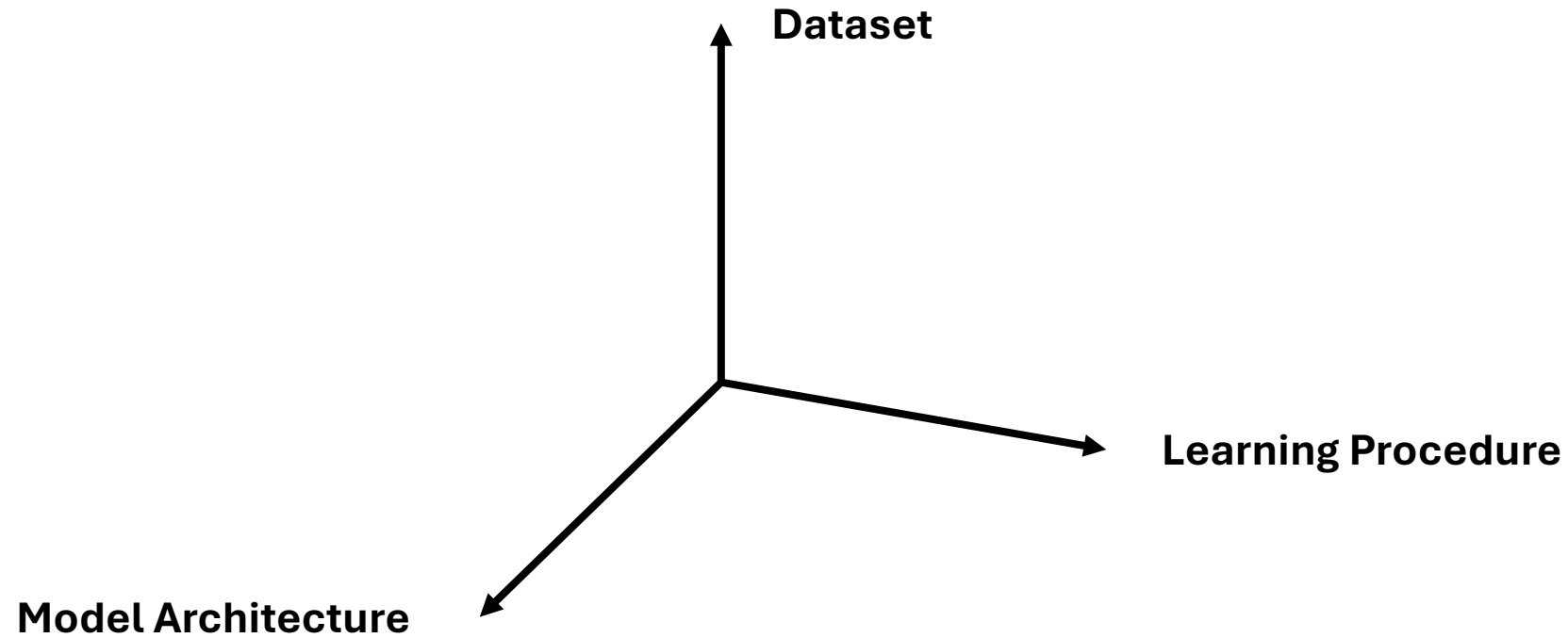


The Course So Far: Synthesis Strategy

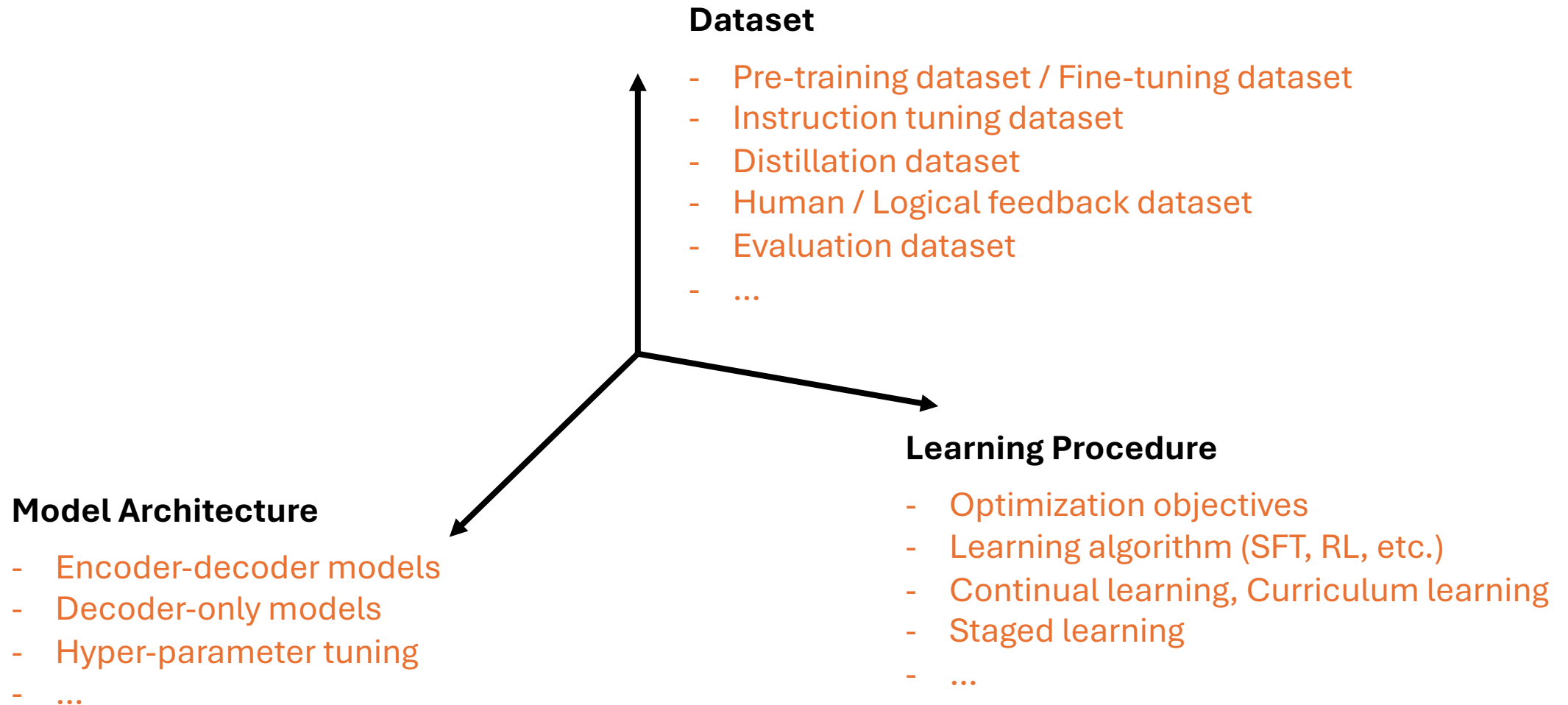
- No prior knowledge
 - Enumerate the entire program space to find the “correct” program
- With prior knowledge: **assumes a good enough language model**
 - We can query language model to write simple programs
 - We can perform **prompting strategies** to search for a solution
 - We can build **agentic framework with tools** to augment the synthesis

How do we obtain a good enough language model?

How to obtain a “good enough” LLM

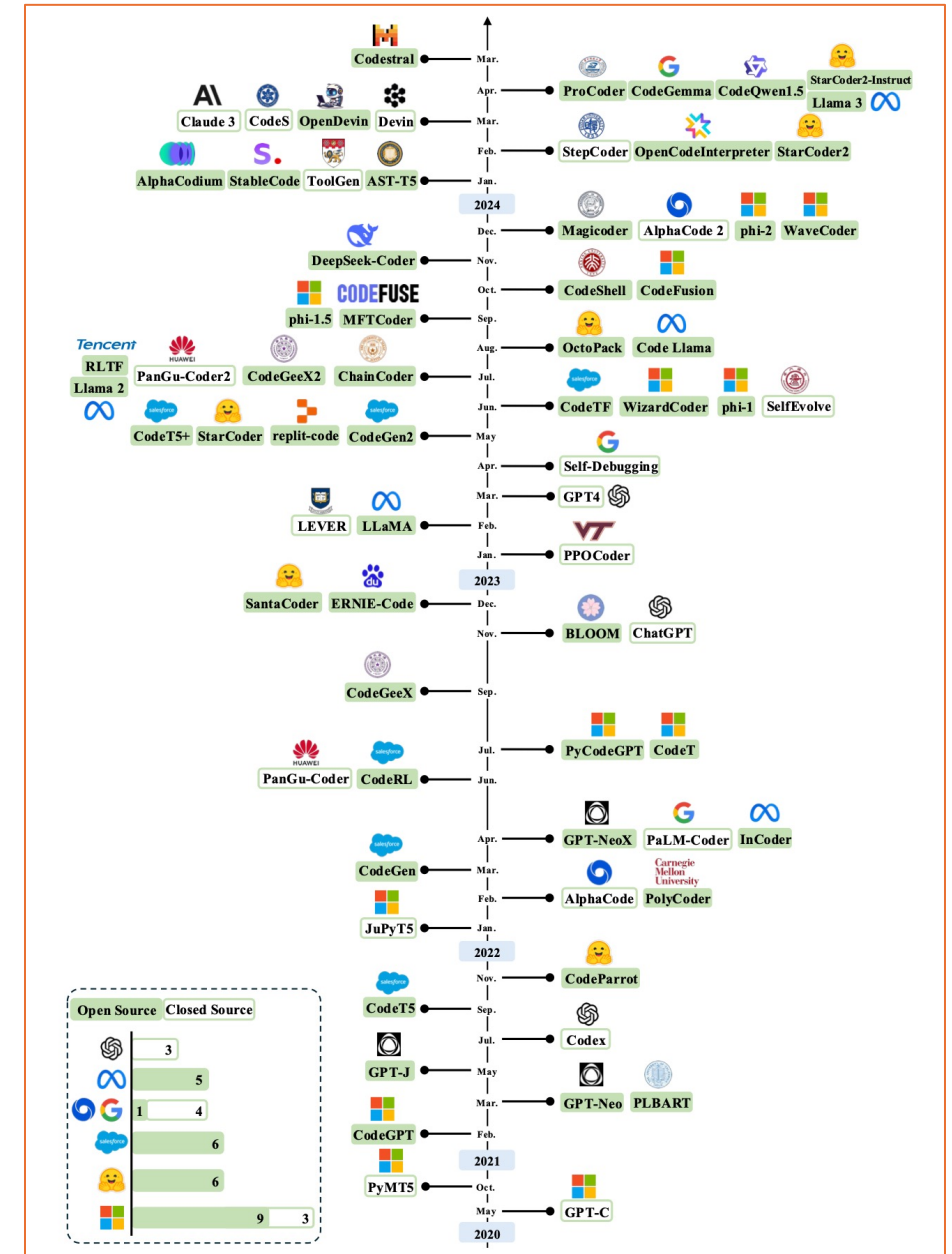


How to obtain a “good enough” LLM



Language Models

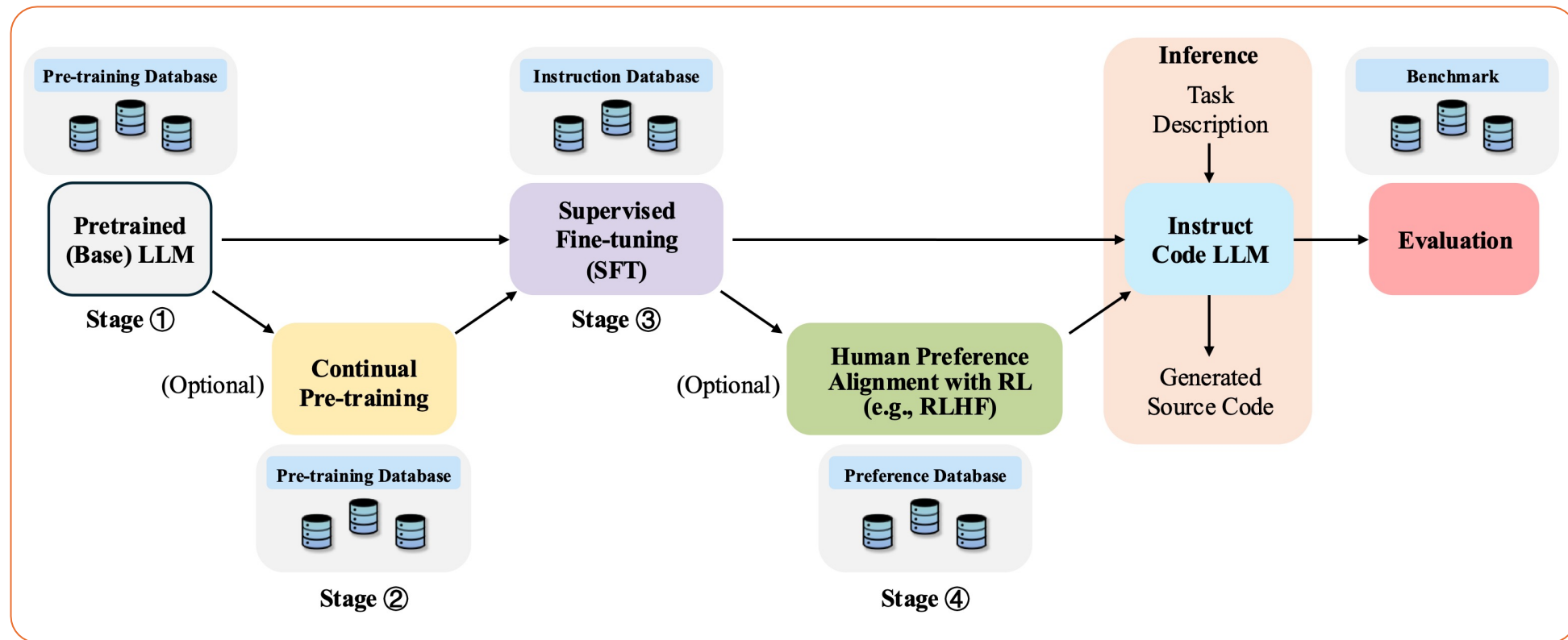
- General purpose ones
 - GPT series (Astra, Luna, ...)
 - Gemini series (3, 4-Argon) / Gemma
 - Claude series (sonnet, opus, fable, ...)
 - Llama series (1, 2, 3, 4)
 - DeepSeek series (v1, v2, v3, v4)
- Specialized for:
 - Reasoning: deepseek-r1
 - Coding: Code Llama, DeepSeek Coder



Learning Objective

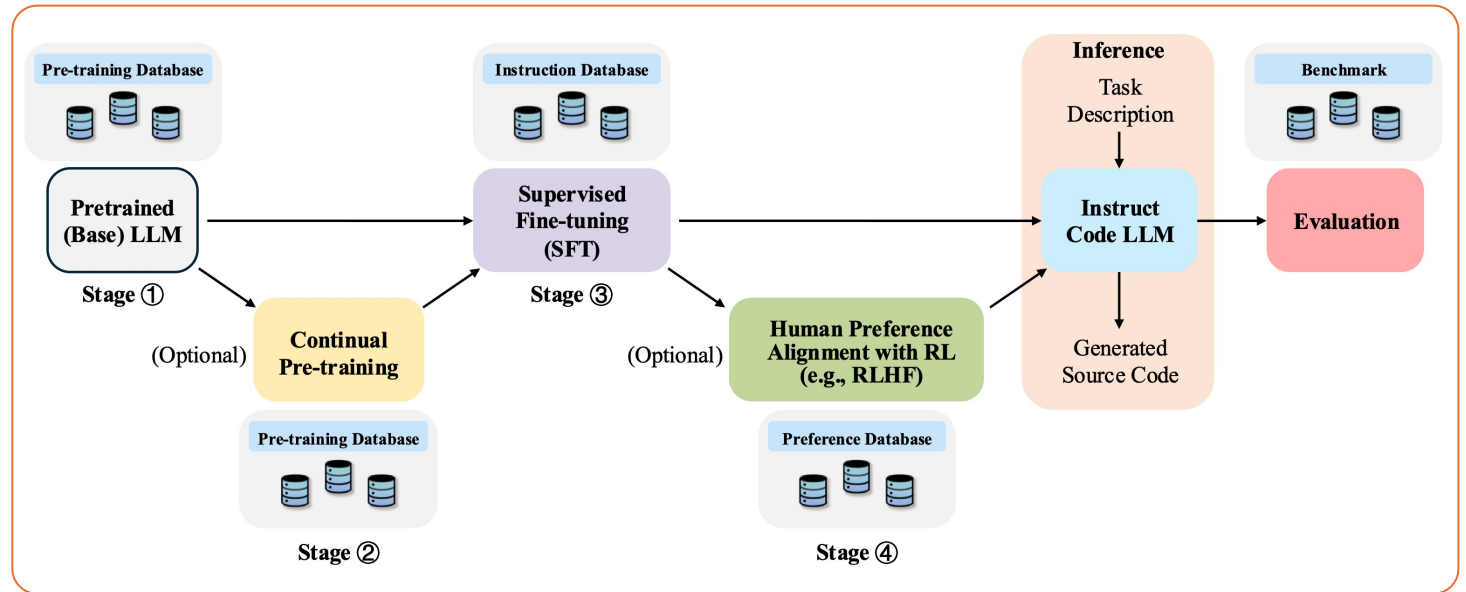
-  This lecture is **NOT** about:
 - Memorizing every model name, size, or configuration
 - Ranking models by “who’s best” or “who wins on benchmark X”
 - Treating architecture, objective, or training stage as absolute recipes
 - Chasing transient leaderboard scores or buzzwords
-  This lecture **IS** about:
 - Grasping the conceptual framework behind how LLMs are trained
 - Developing the skill to **read** new papers, **extract** the key ideas, and **connect** them to broader trends
 - Recognizing trade-offs and design rationales, not just final numbers
 - Building intuition to **anticipate** and **interpret future developments**

High-level Training, Inference, and Evaluation



Today's Agenda

- Pre-training stage
 - Model architecture
 - Pre-training dataset
 - Learning objectives
 - Evaluation dataset
- Special topics
 - Post-training staging
 - Scaling law
 - Hallucination



Model Architecture Case Study: Llama 3



The Llama 3 Herd of Models

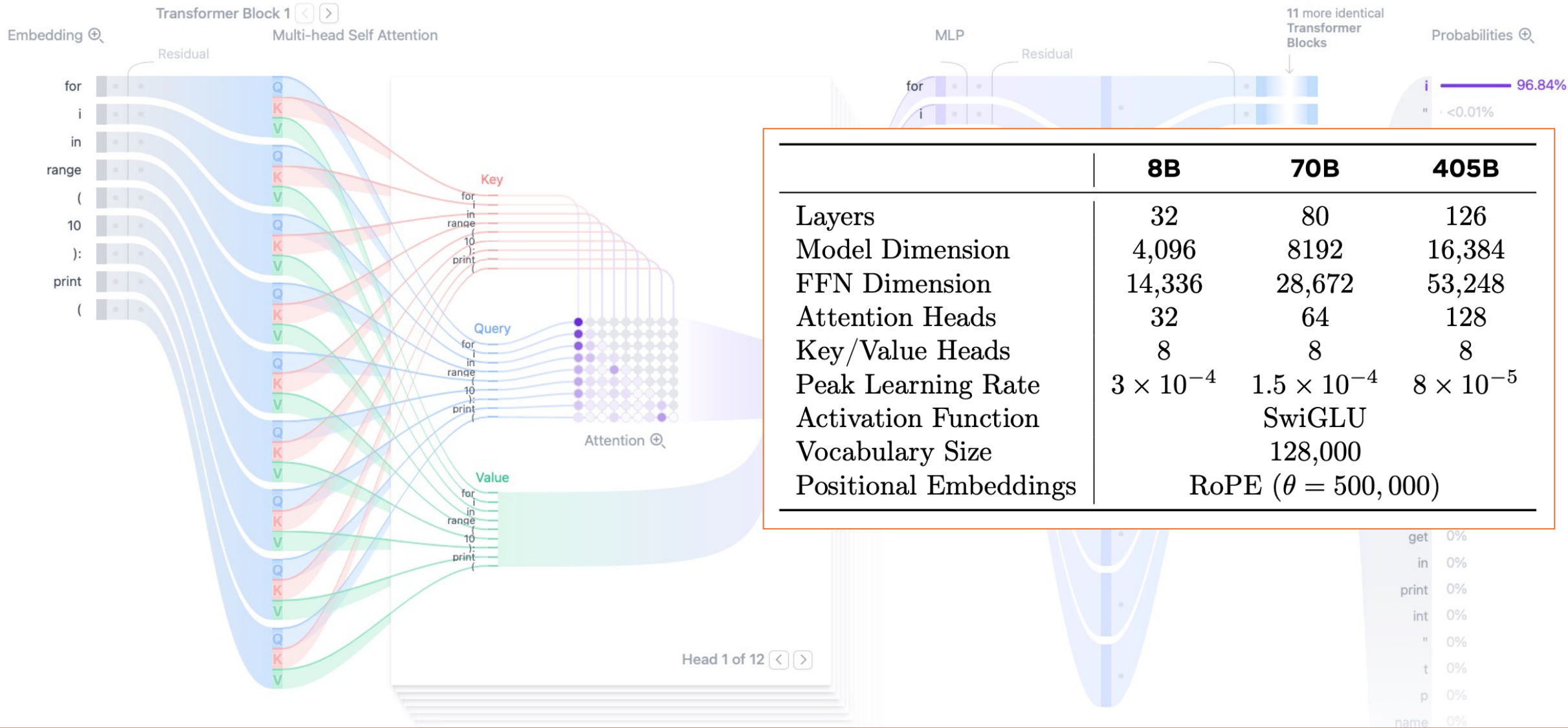
Llama Team, AI @ Meta¹

¹A detailed contributor list can be found in the appendix of this paper.

Modern artificial intelligence (AI) systems are powered by foundation models. This paper presents a new set of foundation models, called Llama 3. It is a herd of language models that natively support multilinguality, coding, reasoning, and tool usage. Our largest model is a dense Transformer with 405B parameters and a context window of up to 128K tokens. This paper presents an extensive empirical evaluation of Llama 3. We find that Llama 3 delivers comparable quality to leading language models such as GPT-4 on a plethora of tasks. We publicly release Llama 3, including pre-trained and post-trained versions of the 405B parameter language model and our Llama Guard 3 model for input and output safety. The paper also presents the results of experiments in which we integrate image, video, and speech capabilities into Llama 3 via a compositional approach. We observe this approach performs competitively with the state-of-the-art on image, video, and speech recognition tasks. The resulting models are not yet being broadly released as they are still under development.

Date: July 23, 2024

Website: <https://llama.meta.com/>



GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

GPT-4 Technical Report

2 Scope and Limitations of this Technical Report

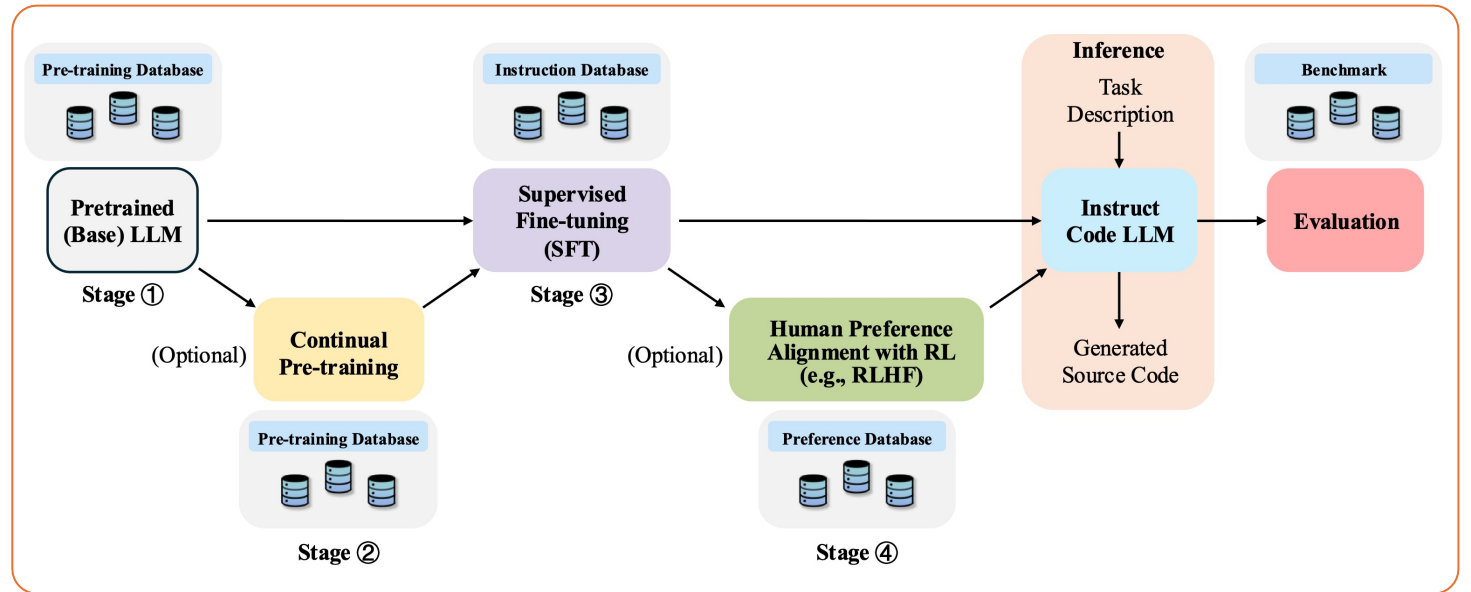
This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.² We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

We report on the capabilities, limitations, and safety properties of GPT-4. We accept input from humans on various bar exams based on alignment and adherence to infrastructure range of performance GPT-4.

Today's Agenda

- Pre-training stage
 - ~~Model architecture~~
 - ~~Pre-training dataset~~
 - Learning objectives
 - Evaluation dataset
- Special topics
 - Post-training staging
 - Scaling law
 - Hallucination



Pre-training: Learning Objectives

- Causal Language Modeling
 - Next token prediction
 - Infilling
 - Multi-token prediction
- Auxiliary pre-training tasks
 - Masked token prediction
 - (Coding) Masked identifier prediction
 - (Coding) Identifier tagging
 - (Coding) Text-code matching
 - (Coding) Text-code contrastive learning

Pre-training: Learning Objectives

- Learning Objective (Machine Learning 101)
 - Loss function $\mathcal{L}(\mathbf{x}; \theta)$ where θ is the model parameter

$$\theta = \operatorname{argmin}_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}(\mathbf{x}; \theta)$$

Pre-training: Learning Objectives

- Learning Objective (Machine Learning 101)
 - Loss function $\mathcal{L}(\mathbf{x}; \theta)$ where θ is the model parameter

$$\theta = \operatorname{argmin}_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}(\mathbf{x}; \theta)$$

- Next-token prediction

$$\mathcal{L}(\mathbf{x}; \theta) = \sum_{i=1}^n -\log P_{\theta}(x_i \mid \mathbf{x}_{<i})$$

Pre-training: Learning Objectives

- Learning Objective (Machine Learning 101)
 - Loss function $\mathcal{L}(\mathbf{x}; \theta)$ where θ is the model parameter

$$\theta = \operatorname{argmin}_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}(\mathbf{x}; \theta)$$

- Next-token prediction

$$\mathcal{L}(\mathbf{x}; \theta) = \sum_{i=1}^n -\log P_{\theta}(x_i \mid \mathbf{x}_{<i})$$

- Example `[for, i, in, range, (, 10, ), :, print, (]`
 $x \leq 11$

Diagram illustrating the input to a sequence model. A sequence of tokens $x_1, x_2, \dots, x_i, \dots, x_n$ is shown. The token x_i is highlighted in orange. The sequence is represented as a vector $P_\theta(x_i | \mathbf{x}_{<i})$.

Pre-training: Learning Objectives

- Next-token prediction
 - Taking prefix $x_{<i}$ and predict the next token x_i
 - But what about code editing happening in the middle?

```
149 impl NodeVisitor<Variable> for LocalTypingContext {
150     fn visit(&mut self, node: &Variable) {
151         // Collect the variable
152         if let Some(local_path: String) = FIRPath::from_ast(path: node.name()).local_path() {
153             self &mut LocalTypingContext
154                 .variables HashMap<String, Vec<NodeLocation>>
155                 .entry(key: local_path) Entry<'_, String, Vec<NodeLocation>>
156                 .or_insert(default: vec![]) &mut Vec<NodeLocation>
157                 .push(node.location().clone());
158         }
159
160         let path = FIRPath::from_ast(node.name());
161
162         // Add the variable constraint to the context
163         self.constraints.push(TypeConstraint::Variable {
164             node: node.location().clone(),
165             variable: FIRPath::from_ast(path: node.name()),
166         });
167     }
168 }
```

Pre-training: Learning Objectives

- Next-token prediction
 - Taking prefix $\mathbf{x}_{<i}$ and predict the next token x_i
 - But what about code editing happening in the middle?
- Infilling
 - Assume prefix $\mathbf{x}_{<i}$ and suffix $\mathbf{x}_{>j}$, predict the middle infill $\mathbf{x}_{i:j}$
 - Idea: **reduce** the problem of **infilling** to next-token prediction



DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence

Daya Guo^{*1}, Qihao Zhu^{*1,2}, Dejian Yang¹, Zhenda Xie¹, Kai Dong¹, Wentao Zhang¹
Guanting Chen¹, Xiao Bi¹, Y. Wu¹, Y.K. Li¹, Fuli Luo¹, Yingfei Xiong², Wenfeng Liang¹

¹DeepSeek-AI

²Key Lab of HCST (PKU), MOE; SCS, Peking University
{zhuqh, guodaya}@deepseek.com

<https://github.com/deepseek-ai/DeepSeek-Coder>

DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence

Daya Guo^{*1}, Qihao Zhu^{*1,2}, Dejian Yang¹, Zhenda Xie¹, Kai Dong¹, Wentao Zhang¹
Guanting Chen¹, Xiao Bi¹, Y. Wu¹, Y.K. Li¹, Fuli Luo¹, Yingfei Xiong², Wenfeng Liang¹

In our implementation, we have introduced three sentinel tokens specifically for this task. For each code file, we initially divide its content into three segments, denoted as f_{pre} , f_{middle} , and f_{suf} . Using the PSM mode, we construct the training example as follows:

$$< | \text{fim_start} | > f_{pre} < | \text{fim_hole} | > f_{suf} < | \text{fim_end} | > f_{middle} < | \text{eos_token} | >$$

We implement the Fill-in-the-Middle (FIM) method at the document level before the packing process, as proposed in the original work by [Bavarian et al. \(2022\)](#). This is done with an FIM rate of 0.5, following the PSM mode.

Infilling / Fill-in-the-Middle (FIM)

```
impl TypeVarAllocator {  
    pub fn new() -> TypeVarAllocator {  
        TypeVarAllocator { next_id: 1 }  
    }  
  
    pub fn next_id(&mut self) -> FIRUnifVarId {  
        let id = FIRUnifVarId(self.next_id);  
        self.next_id += 1;  
        id  
    }  
  
    pub fn reset(&mut self) {  
        self.next_id = 1;  
    }  
}
```

Infilling / Fill-in-the-Middle (FIM)

```
impl TypeVarAllocator {  
    pub fn new() -> TypeVarAllocator {  
        TypeVarAllocator { next_id: 1 }  
    }  
  
    pub fn next_id(&mut self) -> FIRUnifVarId {  
        let id = FIRUnifVarId(self.next_id);  
        self.next_id += 1;  
        id  
    }  
  
    pub fn reset(&mut self) {  
        self.next_id = 1;  
    }  
}
```

Infilling / Fill-in-the-Middle (FIM)

```
impl TypeVarAllocator {  
  pub fn new() -> TypeVarAllocator {  
    TypeVarAllocator { next_id: 1 }  
  }  
  
  pub fn next_id(&mut self) -> FIRUnifVarId {  
    let id = FIRUnifVarId(self.next_id);  
    self.next_id += 1;  
    id  
  }  
  
  pub fn reset(&mut self) {  
    self.next_id = 1;  
  }  
}
```



```
impl TypeVarAllocator {  
  pub fn new() -> TypeVarAllocator {  
    TypeVarAllocator { next_id: 1 }  
  }  
  
  pub fn next_id(&mut self) -> FIRUnifVarId {
```

```
}  
  
  pub fn reset(&mut self) {  
    self.next_id = 1;  
  }  
}
```

```
    let id = FIRUnifVarId(self.next_id);  
    self.next_id += 1;  
    id
```

Infilling / Fill-in-the-Middle (FIM)

```
impl TypeVarAllocator {  
  pub fn new() -> TypeVarAllocator {  
    TypeVarAllocator { next_id: 1 }  
  }  
  
  pub fn next_id(&mut self) -> FIRUnifVarId {  
    let id = FIRUnifVarId(self.next_id);  
    self.next_id += 1;  
    id  
  }  
  
  pub fn reset(&mut self) {  
    self.next_id = 1;  
  }  
}
```



<FIM_START>

```
impl TypeVarAllocator {  
  pub fn new() -> TypeVarAllocator {  
    TypeVarAllocator { next_id: 1 }  
  }  
  
  pub fn next_id(&mut self) -> FIRUnifVarId {
```

<FIM_HOLE>

```
}  
  
  pub fn reset(&mut self) {  
    self.next_id = 1;  
  }  
}
```

<FIM_END>

```
    let id = FIRUnifVarId(self.next_id);  
    self.next_id += 1;  
    id
```

<EOS>

Infilling / Fill-in-the-Middle (FIM)

- A single data-point can be augmented into **multiple** data-points for in-filling
- Suits modern developer workflow nicely:
 - Developer may be working on an existing file
 - Developer wants to change a function or edit a part of the file
- Question:
 - Where do we slice the program?

<FIM_START>

```
impl TypeVarAllocator {  
    pub fn new() -> TypeVarAllocator {  
        TypeVarAllocator { next_id: 1 }  
    }  
  
    pub fn next_id(&mut self) -> FIRUnifVarId {
```

<FIM_HOLE>

```
}  
  
    pub fn reset(&mut self) {  
        self.next_id = 1;  
    }  
}
```

<FIM_END>

```
    let id = FIRUnifVarId(self.next_id);  
    self.next_id += 1;  
    id
```

<EOS>

Code Llama: Open Foundation Models for Code

Baptiste Rozière[†], Jonas Gehring[†], Fabian Gloeckle^{†,*}, Sten Sootla[†], Itai Gat, Xiaoqing Ellen Tan, Yossi Adi[◇], Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, Gabriel Synnaeve[†]

Meta AI

Code Llama: Open Foundation Models for Code

Bapt
Ellen
Kozh
Grat
Louis

2.3 Infilling

Code infilling is the task of predicting the missing part of a program given a surrounding context. Applications include code completion at the cursor's position in code IDEs, type inference and generation of in-code documentation (e.g., docstrings).

We train infilling models following the concept of causal masking (Aghajanyan et al., 2022; Fried et al., 2023), where parts of a training sequence are moved to the end, and the reordered sequence is predicted autoregressively. We train the general-purpose 7B, 13B and 70B models with an infilling objective, following the recommendations of Bavarian et al. (2022). More precisely, we split training documents at the character level into a prefix, a middle part and a suffix with the splitting locations sampled independently from a uniform distribution over the document length. We apply this transformation with a probability of 0.9 and to documents that are not cut across multiple model contexts only. We randomly format half of the splits in the *prefix-suffix-middle* (PSM) format and the other half in the compatible *suffix-prefix-middle* (SPM) format described in Bavarian et al. (2022, App. D). We extend LLAMA 2's tokenizer with four special tokens that mark the beginning of the prefix, the middle part or the suffix, and the end of the infilling span. To limit the distribution shift between autoregressive and infilling training, we suppress the implicit leading space that SentencePiece tokenizers add upon encoding the middle part and the suffix (Kudo & Richardson, 2018). In SPM format, we concatenate the prefix and the middle part before encoding to tokens. Note that our model doesn't encounter split subtokens in the SPM format while it does in the PSM format.

Results on the effect of infilling training on downstream generation tasks and the performance of our infilling models on infilling benchmarks are reported in Section 3.2.

Code Llama: Open Foundation Models for Code

Bapt
Ellen
Kozh
Grat
Louis

2.3 Infilling

Code infilling is the task of predicting the missing part of a program given a surrounding context. Applications include code completion at the cursor's position in code IDEs, type inference and generation of in-code documentation (e.g., docstrings).

We train infilling models following the concept of causal masking (Aghajanyan et al., 2022; Fried et al., 2023), where parts of a training sequence are moved to the end, and the reordered sequence is predicted autoregressively. We train the general-purpose 7B, 13B and 70B models with an infilling objective, following the recommendations of Bavarian et al. (2022). More precisely, we split training documents at the character level into a prefix, a middle part and a suffix with the splitting locations sampled independently from a uniform distribution over the document length. We apply this transformation with a probability of 0.9 and to documents that are not cut across multiple model contexts only. We randomly format half of the splits in the *prefix-suffix-middle* (PSM) format and the other half in the compatible *suffix-prefix-middle* (SPM) format described in Bavarian et al. (2022, App. D). We extend LLAMA 2's tokenizer with four special tokens that mark the beginning of the prefix, the middle part or the suffix, and the end of the infilling span. To limit the distribution shift between autoregressive and infilling training, we suppress the implicit leading space that SentencePiece tokenizers add upon encoding the middle part and the suffix (Kudo & Richardson, 2018). In SPM format, we concatenate the prefix and the middle part before encoding to tokens. Note that our model doesn't encounter split subtokens in the SPM format while it does in the PSM format.

Results on the effect of infilling training on downstream generation tasks and the performance of our infilling models on infilling benchmarks are reported in Section 3.2.

Code Llama: Open Foundation Models for Code

Bapt
Ellen
Kozh
Grat
Louis

2.3 Infilling

Code infilling is the task of predicting the missing part of a program given a surrounding context. Applications include code completion at the cursor's position in code IDEs, type inference and generation of in-code documentation (e.g., docstrings).

We train infilling models following the concept of causal masking (Aghajanyan et al., 2022; Fried et al., 2023), where parts of a training sequence are moved to the end, and the reordered sequence is predicted autoregressively. We train the general-purpose 7B, 13B and 70B models with an infilling objective, following the recommendations of Bavarian et al. (2022). More precisely, we split training documents at the character level into a prefix, a middle part and a suffix with the splitting locations sampled independently from a uniform distribution over the document length. We apply this transformation with a probability of 0.9 and to documents that are not cut across multiple model contexts only. We randomly format half of the splits in the *prefix-suffix-middle* (PSM) format and the other half in the compatible *suffix-prefix-middle* (SPM) format described in Bavarian et al. (2022, App. D). We extend LLAMA 2's tokenizer with four special tokens that mark the beginning of the prefix, the middle part or the suffix, and the end of the infilling span. To limit the distribution shift between autoregressive and infilling training, we suppress the implicit leading space that SentencePiece tokenizers add upon encoding the middle part and the suffix (Kudo & Richardson, 2018). In SPM format, we concatenate the prefix and the middle part before encoding to tokens. Note that our model doesn't encounter split subtokens in the SPM format while it does in the PSM format.

Results on the effect of infilling training on downstream generation tasks and the performance of our infilling models on infilling benchmarks are reported in Section 3.2.

Code Llama: Open Foundation Models for Code

2.3 Infilling

Baptiste
Ellen
Kozhe
Gratta
Louis

Code infilling is the task of predicting the missing part of a program given a surrounding context. Applications include code completion at the cursor's position in code IDEs, type inference and generation of in-code documentation.

We train infilling models (Roziere et al., 2023), where pre-training is done autoregressively on the recommended level into a pre-training distribution to documents to documents to documents the *prefix-suffix* distribution described in Ba mark the beginn distribution shi SentencePiece to SPM format, we doesn't encount

Results on the c models on infill

at, Xiaoqing
pin, Artvom

Model	FIM	Size	HumanEval			MBPP			Test loss
			pass@1	pass@10	pass@100	pass@1	pass@10	pass@100	
CODE LLAMA (w/o LCFT)	✗	7B	33.2%	43.3%	49.9%	44.8%	52.5%	57.1%	0.408
		13B	36.8%	49.2%	57.9%	48.2%	57.4%	61.6%	0.372
CODE LLAMA (w/o LCFT)	✓	7B	33.6%	44.0%	48.8%	44.2%	51.4%	55.5%	0.407
		13B	36.2%	48.3%	54.6%	48.0%	56.8%	60.8%	0.373
Absolute gap	✗ - ✓	7B	-0.4%	-0.7%	1.1%	0.6%	1.1%	1.6%	0.001
		13B	0.7%	0.9%	3.3%	0.2%	0.6%	0.8%	-0.001

Table 5: **Comparison of models with and without FIM training.** pass@1, pass@10 and pass@100 scores on HumanEval and MBPP evaluated at temperature 0.1 for models trained with and without infilling (FIM) objective. Infilling training incurs no cost on autoregressive test set loss, but a small cost on HumanEval and MBPP pass@k metrics that is aggravated at higher sample counts k . The models are compared prior to long context fine-tuning (LCFT).

Pre-training: Learning Objectives

- Next-token prediction
 - Taking prefix $\mathbf{x}_{<i}$ and predict the next token x_i
 - But what about code editing happening in the middle?
- Infilling
 - Assume prefix $\mathbf{x}_{<i}$ and suffix $\mathbf{x}_{>j}$, predict the middle infill $\mathbf{x}_{i:j}$
 - Idea: **reduce** the problem of **infilling** to next-token prediction
- Multi-token prediction (MTP)
 - Taking prefix $\mathbf{x}_{\leq i}$ and predict the next k tokens $x_{i+1}, \dots, x_{i+k}$
 - Idea: have multiple prediction heads each predicting the next i -th token

Multi-token Prediction

Better & Faster Large Language Models via Multi-token Prediction

Fabian Gloeckle^{*12} **Badr Youbi Idrissi**^{*13} **Baptiste Rozière**¹ **David Lopez-Paz**⁺¹ **Gabriel Synnaeve**⁺¹

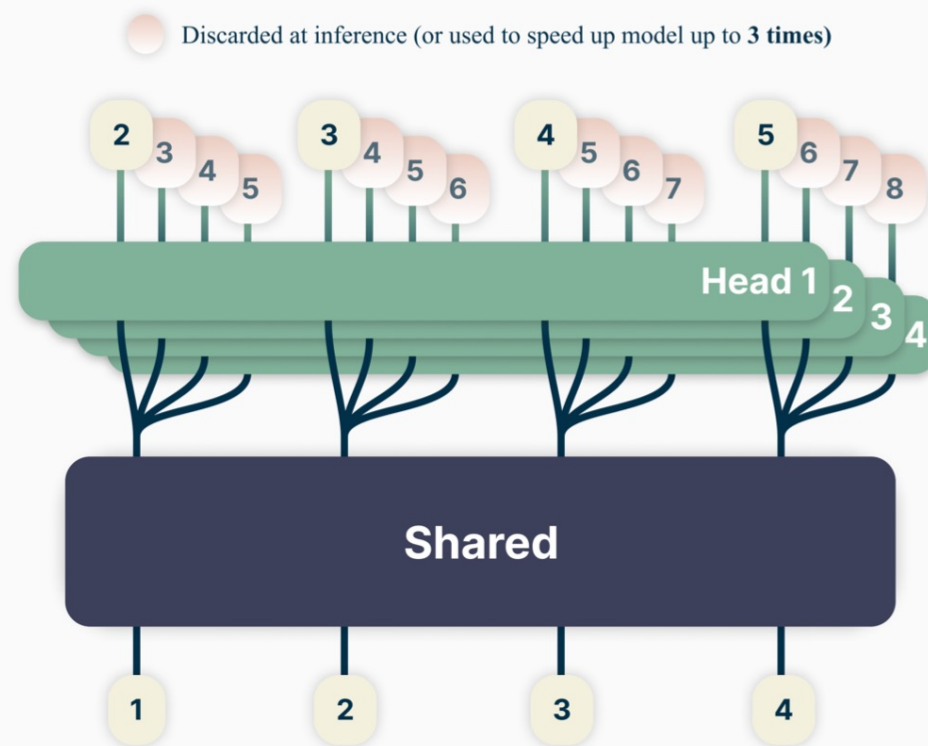
Multi-token Prediction

Better & Faster LLMs

Fabian Gloeckle^{*12} Badr Yoonis

4-token
targets

Inputs



Multi-token Pre

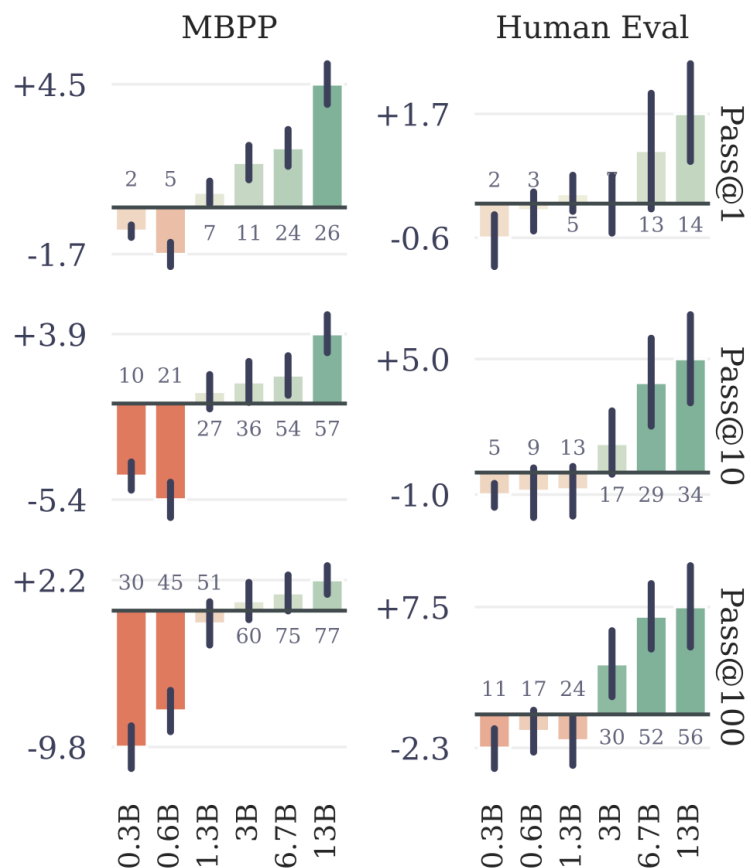
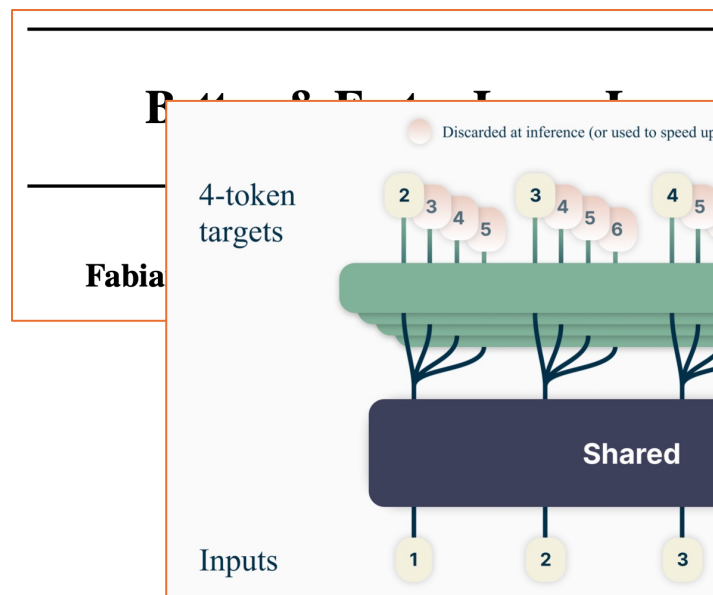


Figure 3: **Results of n -token prediction models on MBPP by model size.** We train models of six sizes in the range or 300M to 13B total parameters on code, and evaluate pass@1,10,100 on the MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021) benchmark with 1000 samples. Multi-token prediction models are worse than the baseline for small model sizes, but outperform the baseline at scale. Error bars are confidence intervals of 90% computed with bootstrapping over dataset samples.



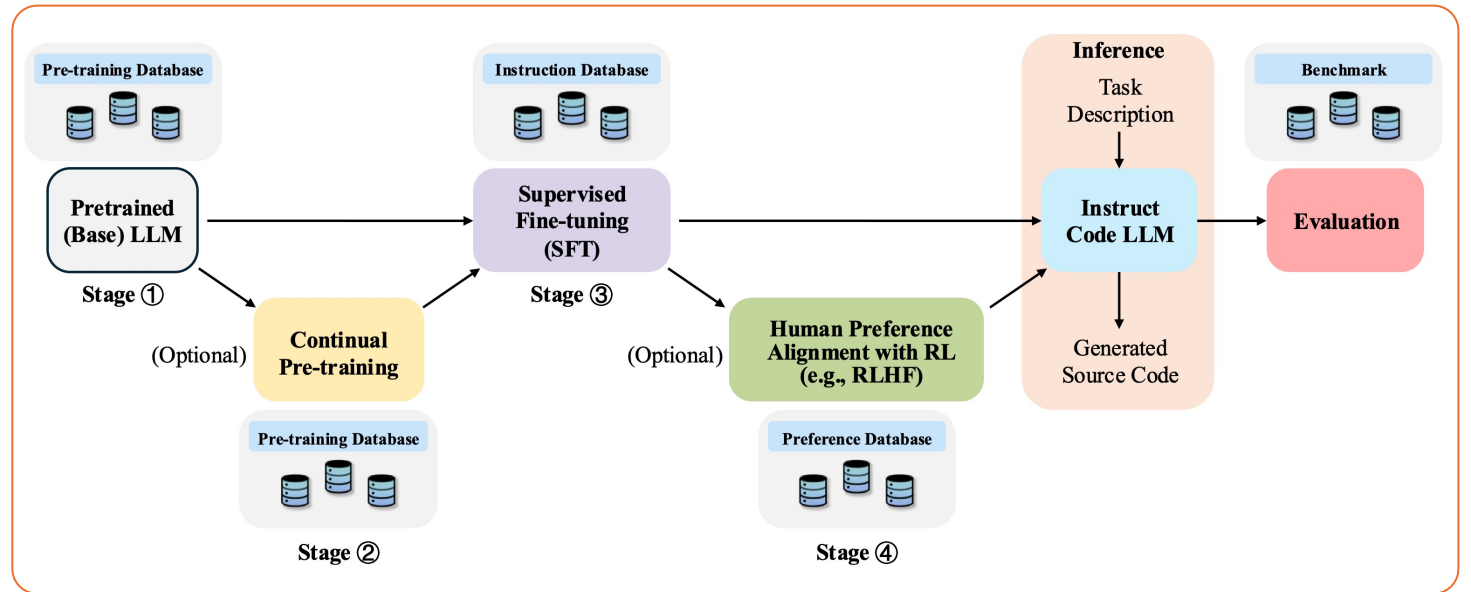
DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

Multi-Token Prediction. As DeepSeek-V3, DeepSeek-V4 series also set MTP modules and objectives. Given that the MTP strategy has been validated in DeepSeek-V3, we adopt the same strategy for DeepSeek-V4 series without modification.

Today's Agenda

- Pre-training stage
 - ~~Model architecture~~
 - ~~Pre-training dataset~~
 - ~~Learning objectives~~
 - Evaluation dataset
- Special topics
 - Post-training staging
 - Scaling law
 - Hallucination



Evaluation Benchmark

- Coding benchmarks can be used to evaluate LLMs' abilities

Category	Benchmark	Llama 3 8B	Gemma 2 9B	Mistral 7B	Llama 3 70B	Mixtral 8x22B	GPT 3.5 Turbo	Llama 3 405B	Nemotron 4 340B	GPT-4 (0125)	GPT-4o	Claude 3.5 Sonnet
General	MMLU _(5-shot)	69.4	72.3	61.1	83.6	76.9	70.7	87.3	82.6	85.1	89.1	89.9
	MMLU _(0-shot, CoT)	73.0	72.3 [△]	60.5	86.0	79.9	69.8	88.6	78.7 [△]	85.4	88.7	88.3
	MMLU-Pro _(5-shot, CoT)	48.3	–	36.9	66.4	56.3	49.2	73.3	62.7	64.8	74.0	77.0
	IFEval	80.4	73.6	57.6	87.5	72.7	69.9	88.6	85.1	84.3	85.6	88.0
Code	HumanEval _(0-shot)	72.6	54.3	40.2	80.5	75.6	68.0	89.0	73.2	86.6	90.2	92.0
	MBPP EvalPlus _(0-shot)	72.8	71.7	49.5	86.0	78.6	82.0	88.6	72.8	83.6	87.8	90.5
Math	GSM8K _(8-shot, CoT)	84.5	76.7	53.2	95.1	88.2	81.6	96.8	92.3 [◇]	94.2	96.1	96.4 [◇]
	MATH _(0-shot, CoT)	51.9	44.3	13.0	68.0	54.1	43.1	73.8	41.1	64.5	76.6	71.1
Reasoning	ARC Challenge _(0-shot)	83.4	87.6	74.2	94.8	88.7	83.7	96.9	94.6	96.4	96.7	96.7
	GPQA _(0-shot, CoT)	32.8	–	28.8	46.7	33.3	30.8	51.1	–	41.4	53.6	59.4
Tool use	BFCL	76.1	–	60.4	84.8	–	85.9	88.5	86.5	88.3	80.5	90.2
	Nexus	38.5	30.0	24.7	56.7	48.5	37.2	58.7	–	50.3	56.1	45.7
Long context	ZeroSCROLLS/QuALITY	81.0	–	–	90.5	–	–	95.2	–	95.2	90.5	90.5
	InfiniteBench/En.MC	65.1	–	–	78.2	–	–	83.4	–	72.1	82.5	–
	NIH/Multi-needle	98.8	–	–	97.5	–	–	98.1	–	100.0	100.0	90.8
Multilingual	MGSM _(0-shot, CoT)	68.9	53.2	29.9	86.9	71.1	51.4	91.6	–	85.9	90.5	91.6

Evaluation Benchmark

Scenario	Benchmark	Size	#PL	Date	Link
General	HumanEval [48]	164	Python	2021-07	https://huggingface.co/datasets/openai_humanEval
	HumanEval+ [162]	164	Python	2023-05	https://huggingface.co/datasets/evalplus/humanevalplus
	HumanEvalPack [187]	164	6	2023-08	https://huggingface.co/datasets/bigcode/humanevalpack
	MBPP [17]	974	Python	2021-08	https://huggingface.co/datasets/mbpp
	MBPP+ [162]	378	Python	2023-05	https://huggingface.co/datasets/evalplus/mbppplus
	CoNaLa [297]	596.88K	Python	2018-05	https://huggingface.co/datasets/neulab/conala
	Spider [300]	8,034	SQL	2018-09	https://huggingface.co/datasets/xlangai/spider
	CONCODE [113]	104K	Java	2018-08	https://huggingface.co/datasets/AhmedSSoliman/CONCOD
	ODEX [273]	945	Python	2022-12	https://huggingface.co/datasets/neulab/odex
	CoderEval [299]	460	Python, Java	2023-02	https://github.com/CoderEval/CoderEval
	ReCode [263]	1,138	Python	2022-12	https://github.com/amazon-science/recode
	StudentEval [19]	1,749	Python	2023-06	https://huggingface.co/datasets/wellesley-easel/StudentEval
	BigCodeBench [333]	1,140	Python	2024-06	https://huggingface.co/datasets/bigcode/bigcodebench
Competitions	ClassEval [72]	100	Python	2023-08	https://huggingface.co/datasets/FudanSELab/ClassEval
	NaturalCodeBench [314]	402	Python, Java	2024-05	https://github.com/THUDM/NaturalCodeBench
	APPS [95]	10,000	Python	2021-05	https://huggingface.co/datasets/codeparrot/apps
Competitions	CodeContests [151]	13,610	C++, Python, Java	2022-02	https://huggingface.co/datasets/deepmind/code_contests
	LiveCodeBench [188]	713 Updating	Python	2024-03	https://github.com/LiveCodeBench/LiveCodeBench
Data Science	DSP [41]	1,119	Python	2022-01	https://github.com/microsoft/DataScienceProblems
	DS-1000 [136]	1,000	Python	2022-11	https://huggingface.co/datasets/xlangai/DS-1000
	ExeDS [107]	534	Python	2022-11	https://github.com/Jun-jie-Huang/ExeDS

Multilingual	MBXP [16]	12.4K	13	2022-10	https://huggingface.co/datasets/mxeval/mbxp
	Multilingual HumanEval [16]	1.9K	12	2022-10	https://huggingface.co/datasets/mxeval/multi-humaneval
	HumanEval-X [321]	820	Python, C++, Java, JavaScript, Go	2023-03	https://huggingface.co/datasets/THUDM/humaneval-x
	MultiPL-E [39]	161	18	2022-08	https://huggingface.co/datasets/nuprl/MultiPL-E
Reasoning	xCodeEval [128]	5.5M	11	2023-03	https://github.com/ntunlp/xCodeEval
	MathQA-X [16]	5.6K	Python, Java, JavaScript	2022-10	https://huggingface.co/datasets/mxeval/mathqa-x
	MathQA-Python [17]	23,914	Python	2021-08	https://github.com/google-research/google-research
	GSM8K [58]	8.5K	Python	2021-10	https://huggingface.co/datasets/gsm8k
	GSM-HARD [79]	1.32K	Python	2022-11	https://huggingface.co/datasets/reasoning-machines/gsm-hard
Repository	CRUXEval [82]	800	Python	2024-01	https://huggingface.co/datasets/cruxeval-org/cruxeval
	RepoEval [309]	3,573	Python, Java	2023-03	https://paperswithcode.com/dataset/repoeval
	Stack-Repo [239]	200	Java	2023-06	https://huggingface.co/datasets/RepoFusion/Stack-Repo
	Repobench [167]	27k	Python, Java	2023-01	https://github.com/Leolty/repobench
	EvoCodeBench [144]	275	Python	2024-03	https://huggingface.co/datasets/LJ0815/EvoCodeBench
	SWE-bench [123]	2,294	Python	2023-10	https://huggingface.co/datasets/princeton-nlp/SWE-bench
	CrossCodeEval [68]	10K	Python, Java, TypeScript, C#	2023-10	https://github.com/amazon-science/cceval
Repository	SketchEval [308]	20,355	Python	2024-03	https://github.com/nl2code/codes

Evaluation Benchmark: HumanEval

Evaluating Large Language Models Trained on Code

Mark Chen^{*1} Jerry Tworek^{*1} Heewoo Jun^{*1} Qiming Yuan^{*1} Henrique Ponde de Oliveira Pinto^{*1}
Jared Kaplan^{*2} Harri Edwards¹ Yuri Burda¹ Nicholas Joseph² Greg Brockman¹ Alex Ray¹ Raul Puri¹
Gretchen Krueger¹ Michael Petrov¹ Heidy Khlaaf³ Girish Sastry¹ Pamela Mishkin¹ Brooke Chan¹
Scott Gray¹ Nick Ryder¹ Mikhail Pavlov¹ Alethea Power¹ Lukasz Kaiser¹ Mohammad Bavarian¹
Clemens Winter¹ Philippe Tillet¹ Felipe Petroski Such¹ Dave Cummings¹ Matthias Plappert¹
Fotios Chantzis¹ Elizabeth Barnes¹ Ariel Herbert-Voss¹ William Hebggen Guss¹ Alex Nichol¹ Alex Paino¹
Nikolas Tezak¹ Jie Tang¹ Igor Babuschkin¹ Suchir Balaji¹ Shantanu Jain¹ William Saunders¹
Christopher Hesse¹ Andrew N. Carr¹ Jan Leike¹ Josh Achiam¹ Vedant Misra¹ Evan Morikawa¹
Alec Radford¹ Matthew Knight¹ Miles Brundage¹ Mira Murati¹ Katie Mayer¹ Peter Welinder¹
Bob McGrew¹ Dario Amodei² Sam McCandlish² Ilya Sutskever¹ Wojciech Zaremba¹

Evaluation

Evaluati

Mark Chen ^{*1} **Jerry Two**
Jared Kaplan ^{*2} **Harri Edwa**
Gretchen Krueger ¹ **Micha**
Scott Gray ¹ **Nick Ryder** ¹
Clemens Winter ¹ **Phil**
Fotios Chantzis ¹ **Elizabeth B**
Nikolas Tezak ¹ **Jie Tan**
Christopher Hesse ¹ **And**
Alec Radford ¹ **Matthew**
Bob McGrew ¹ **Dar**

Datasets:

openai/openai_humaneval

like
 340

Follow

OpenAI

23.3k

Modalities:

Text

Formats:

parquet

Languages:

English

Size:

< 1K

ArXiv:

arxiv:2107.03374

Tags:

code-generation

Libraries:

Dataset card

Data Studio

Files and versions

 xet

Community

7

Dataset Viewer

Auto-converted to Parquet

</> API

Embed

Data Studio

Split (1)

test · 164 rows

▼

Search this dataset

task_id string · lengths	prompt string · lengths	canonical_solution string · lengths	test string · lengths
HumanEval/0	from typing import List def has_close_elements(numbers: List[float],...	for idx, elem in enumerate(numbers): for idx2, elem2 in enumerate(numbers): if idx !=...	METADATA = { 'au' def check(candid
HumanEval/1	from typing import List def separate_paren_groups(paren_string: str) ->...	result = [] current_string = [] current_depth = 0 for c in paren_string: if...	METADATA = { 'au' def check(candid
HumanEval/2	def truncate_number(number: float) -> float: """ Given a positive floating point number, it can be...	return number % 1.0	METADATA = { 'au' def check(candid
HumanEval/3	from typing import List def below_zero(operations: List[int]) -> bool: """ You're given a list of...	balance = 0 for op in operations: balance += op if balance < 0: return True return False	METADATA = { 'au' def check(candid
HumanEval/4	from typing import List def mean_absolute_deviation(numbers: List[float]) ->...	mean = sum(numbers) / len(numbers) return sum(abs(x - mean) for x in numbers) /...	METADATA = { 'au' def check(candid
HumanEval/5	from typing import List def intersperse(numbers: List[int], delimiter: int) -> List[int]: """...	if not numbers: return [] result = [] for n in numbers[:-1]: result.append(n)...	METADATA = { 'au' def check(candid

< Previous

1

2

Next >

Evaluation Benchmark: HumanEval

Evaluating Large Language Models Trained on Code

Datasets: openai/openai_humaneval 340 Follow OpenAI 23.3k

task_id string · lengths	prompt string · lengths	canonical_solution string · lengths	test string · lengths	entry_point string · lengths
11-12 6.1%	240-365 24.4%	186-271 18.9%	455-624 18.9%	16-19 10.4%
HumanEval/0	<pre>from typing import List def has_close_elements(numbers: List[float], threshold: float) -> bool: """ Check if in given list of numbers, are any two numbers closer to each other than given threshold. >>> has_close_elements([1.0, 2.0, 3.0], 0.5) False >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3) True """</pre>	<pre>for idx, elem in enumerate(numbers): for idx2, elem2 in enumerate(numbers): if idx != idx2: distance = abs(elem - elem2) if distance < threshold: return True return False</pre>	<pre>METADATA = { 'author': 'jt', 'dataset': 'test' } def check(candidate): assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.3) == True assert candidate([1.0, 2.0, 3.9, 4.0, 5.0, 2.2], 0.05) == False assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.95) == True assert candidate([1.0, 2.0, 5.9, 4.0, 5.0], 0.8) == False assert candidate([1.0, 2.0, 3.0, 4.0, 5.0, 2.0], 0.1) == True assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 1.0) == True assert candidate([1.1, 2.2, 3.1, 4.1, 5.1], 0.5) == False</pre>	has_close_elements

GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

GPT-4 Tech

OpenAI

Abstract

We report the development of GPT-4, a large language model that can accept image and text inputs and produce text outputs that are comparable to human performance in many real-world scenarios. GPT-4 is trained on various professional and academic benchmarks, including the bar exam with a score around the top 10% of human performance. The model is based on a pre-trained model pre-trained to predict the next token in a sequence. The alignment process results in improved performance and adherence to desired behavior. A core component of the infrastructure and optimization methodology is the use of a wide range of scales. This allowed us to achieve performance based on models trained with GPT-4.

	GPT-4 Evaluated few-shot	GPT-3.5 Evaluated few-shot	LM SOTA Best external LM evaluated few-shot	SOTA Best external model (incl. benchmark-specific tuning)
MMLU [49] Multiple-choice questions in 57 subjects (professional & academic)	86.4% 5-shot	70.0% 5-shot	70.7% 5-shot U-PaLM [50]	75.2% 5-shot Flan-PaLM [51]
HellaSwag [52] Commonsense reasoning around everyday events	95.3% 10-shot	85.5% 10-shot	84.2% LLaMA (validation set) [28]	85.6 ALUM [53]
AI2 Reasoning Challenge (ARC) [54] Grade-school multiple choice science questions. Challenge-set.	96.3% 25-shot	85.2% 25-shot	85.2% 8-shot PaLM [55]	86.5% ST-MOE [18]
WinoGrande [56] Commonsense reasoning around pronoun resolution	87.5% 5-shot	81.6% 5-shot	85.1% 5-shot PaLM [3]	85.1% 5-shot PaLM [3]
HumanEval [43] Python coding tasks	67.0% 0-shot	48.1% 0-shot	26.2% 0-shot PaLM [3]	65.8% CodeT + GPT-3.5 [57]
DROP [58] (F1 score) Reading comprehension & arithmetic.	80.9 3-shot	64.1 3-shot	70.8 1-shot PaLM [3]	88.4 QDGAT [59]
GSM-8K [60] Grade-school mathematics questions	92.0%* 5-shot chain-of-thought	57.1% 5-shot	58.8% 8-shot Minerva [61]	87.3% Chinchilla + SFT+ORM-RL, ORM reranking [62]

Table 2. Performance of GPT-4 on academic benchmarks. We compare GPT-4 alongside the best SOTA (with benchmark-specific training) and the best SOTA for an LM evaluated few-shot. GPT-4 outperforms existing LMs on all benchmarks, and beats SOTA with benchmark-specific training on all datasets except DROP. For each task we report GPT-4’s performance along with the few-shot method used to evaluate. For GSM-8K, we included part of the training set in the GPT-4 pre-training mix (see Appendix E), and we use chain-of-thought prompting [11] when evaluating. For multiple-choice questions, we present all answers (ABCD) to the model and ask it to choose the letter of the answer, similarly to how a human would solve such a problem.

GPT-4 Tech

OpenAI

Abstract

We report the development of GPT-4, a large language model that can accept image and text inputs and produce text outputs that are comparable to human performance in many real-world scenarios. GPT-4 is trained on various professional and academic benchmarks, including the bar exam with a score around the top 10% of human performance. The model is based on a pre-trained model pre-trained to predict the next token in a sequence. The alignment process results in improved performance and adherence to desired behavior. A core component of the infrastructure and optimization methodology is the use of a wide range of scales. This allowed us to achieve performance based on models trained with GPT-4.

	GPT-4 Evaluated few-shot	GPT-3.5 Evaluated few-shot	LM SOTA Best external LM evaluated few-shot	SOTA Best external model (incl. benchmark-specific tuning)
MMLU [49] Multiple-choice questions in 57 subjects (professional & academic)	86.4% 5-shot	70.0% 5-shot	70.7% 5-shot U-PaLM [50]	75.2% 5-shot Flan-PaLM [51]
HellaSwag [52] Commonsense reasoning around everyday events	95.3% 10-shot	85.5% 10-shot	84.2% LLaMA (validation set) [28]	85.6 ALUM [53]
AI2 Reasoning Challenge (ARC) [54] Grade-school multiple choice science questions. Challenge-set.	96.3% 25-shot	85.2% 25-shot	85.2% 8-shot PaLM [55]	86.5% ST-MOE [18]
WinoGrande [56] Commonsense reasoning around pronoun resolution	87.5% 5-shot	81.6% 5-shot	85.1% 5-shot PaLM [3]	85.1% 5-shot PaLM [3]
HumanEval [43] Python coding tasks	67.0% 0-shot	48.1% 0-shot	26.2% 0-shot PaLM [3]	65.8% CodeT + GPT-3.5 [57]
DROP [58] (F1 score) Reading comprehension & arithmetic.	80.9 3-shot	64.1 3-shot	70.8 1-shot PaLM [3]	88.4 QDGAT [59]
GSM-8K [60] Grade-school mathematics questions	92.0%* 5-shot chain-of-thought	57.1% 5-shot	58.8% 8-shot Minerva [61]	87.3% Chinchilla + SFT+ORM-RL, ORM reranking [62]

Table 2. Performance of GPT-4 on academic benchmarks. We compare GPT-4 alongside the best SOTA (with benchmark-specific training) and the best SOTA for an LM evaluated few-shot. GPT-4 outperforms existing LMs on all benchmarks, and beats SOTA with benchmark-specific training on all datasets except DROP. For each task we report GPT-4’s performance along with the few-shot method used to evaluate. For GSM-8K, we included part of the training set in the GPT-4 pre-training mix (see Appendix E), and we use chain-of-thought prompting [11] when evaluating. For multiple-choice questions, we present all answers (ABCD) to the model and ask it to choose the letter of the answer, similarly to how a human would solve such a problem.

Evaluation Benchmark: MBPP

Program Synthesis with Large Language Models

Jacob Austin*

Augustus Odena*

Maxwell Nye[†]

Maarten Bosma

Henryk Michalewski

David Dohan

Ellen Jiang

Carrie Cai

Michael Terry

Quoc Le

Charles Sutton

Google Research

* denotes equal contribution

jaaustin@google.com, augustusodena@google.com

Program Sy

Jacob

Maxwell Nye[†] Maarten Bos

Michael Terry

jaausti

Datasets:
 evalplus/
 mbppplus

like 14
 Follow EvalPlus 17

Modalities: Text
 Formats: parquet
 Size: <1K
 Libraries: Datasets pandas Croissant +1
 License: apache-2.0

Dataset card
 Data Studio
 Files and versions xet
 Community 1

Dataset Viewer

Auto-converted to Parquet
 API
 Embed
 Data Studio

Split (1)
 test · 378 rows

Search this dataset

task_id	code	prompt	source_file	test_imports	test_list
int64	string · lengths	string · lengths	string · classes	sequence · lengths	sequence · lengths
2	def similar_elements(test_tup1, test_tup2): return...	Write a function to find the share...	Benchmark Questions...	[]	["assert set(simi...
3	import math def is_not_prime(n): if n == 1: return True for i in...	Write a python function to...	Benchmark Questions...	[]	["assert is_not_p...
4	import heapq as hq def heap_queue_largest(nums: list,n...	Write a function to find the n...	Benchmark Questions...	[]	["assert heap_que...
6	def is_Power_Of_Two(x: int): return x > 0 and (x & (x - 1))...	Write a python function to check...	Benchmark Questions...	[]	["assert differ_A...
7	import re def find_char_long(text): return...	Write a function to find all words...	Benchmark Questions...	[]	["assert set(find...
8	def square_nums(nums): return [i**2 for i in nums]	Write a function to find squares o...	Benchmark Questions...	[]	["assert square_n...

< Previous
 1
 2
 3
 4
 Next >

Evaluation Benchmark: MBPP

Program Synthesis with Large Language Models

Datasets: evalplus/mbppplus like 14 Follow EvalPlus 17

Modalities: Text Formats: parquet Size: <1K Libraries: Datasets pandas Croissant +1 License: apache-2.0

Dataset card Data Studio Files and versions xet Community

task_id	code	prompt	source_file	test_imports	test_list
int64	string · lengths	string · lengths	string · classes	list · lengths	list · lengths
 2..82 9.8%	 33..153 79.6%	 39..77 41.8%	 Benchmark ... 63%	 0..1 97.4%	 3..4 92.6%
task_id int64 Split (1) test · 378 Search task_id int64 2	2 <pre>def similar_elements(test_tup1, test_tup2): return tuple(set(test_tup1) & set(test_tup2))</pre>	Write a function to find the shared elements from the given two lists.	Benchmark Question s Verification V2. ipynb	[]	["assert set(similar_elements((3, 4, 5, 6), (5, 7, 4, 10))) == set((4, 5))", "assert set(similar_elements((1, 2, 3, 4), (5, 4, 3, 7))) == set((3, 4))", "assert set(similar_elements((11, 12, 14, 13), (17, 15, 14, 13))) == set((13, 14))"]

Code Llama: Open Foundation Models for Code

2.3 Infilling

Baptiste
Ellen
Kozhe
Gratta
Louis

Code infilling is the task of predicting the missing part of a program given a surrounding context. Applications include code completion at the cursor’s position in code IDEs, type inference and generation of in-code documentation.

We train infilling models (Roziere et al., 2023), where pre-training is done autoregressively on the recommended level into a pre-training distribution to documents to documents to documents the *prefix-suffix* distribution described in Ba mark the beginn distribution shi SentencePiece to SPM format, we doesn’t encount

Results on the c models on infill

at, Xiaoqing
pin, Artvom

Model	FIM	Size	HumanEval			MBPP			Test loss
			pass@1	pass@10	pass@100	pass@1	pass@10	pass@100	
CODE LLAMA (w/o LCFT)	✗	7B	33.2%	43.3%	49.9%	44.8%	52.5%	57.1%	0.408
		13B	36.8%	49.2%	57.9%	48.2%	57.4%	61.6%	0.372
CODE LLAMA (w/o LCFT)	✓	7B	33.6%	44.0%	48.8%	44.2%	51.4%	55.5%	0.407
		13B	36.2%	48.3%	54.6%	48.0%	56.8%	60.8%	0.373
Absolute gap	✗ - ✓	7B	−0.4%	−0.7%	1.1%	0.6%	1.1%	1.6%	0.001
		13B	0.7%	0.9%	3.3%	0.2%	0.6%	0.8%	−0.001

Table 5: **Comparison of models with and without FIM training.** pass@1, pass@10 and pass@100 scores on HumanEval and MBPP evaluated at temperature 0.1 for models trained with and without infilling (FIM) objective. Infilling training incurs no cost on autoregressive test set loss, but a small cost on HumanEval and MBPP pass@k metrics that is aggravated at higher sample counts k . The models are compared prior to long context fine-tuning (LCFT).



EvaLPlus Leaderboard



EvaLPlus evaluates AI Coders with rigorous tests.



News: Beyond correctness, how's their code efficiency? Checkout [EvalPerf!](#)



GITHUB

PAPER

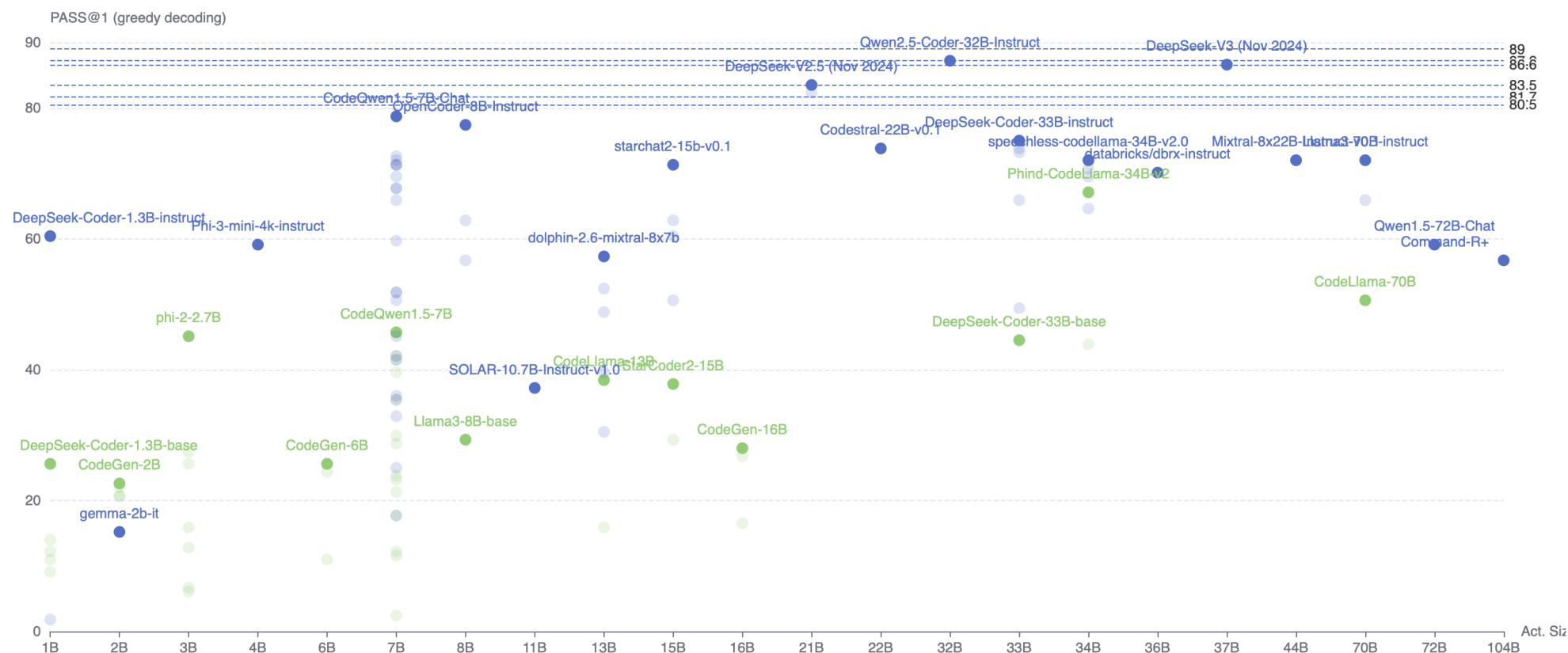
NEURIPS'23

HumanEval

MBPP

Average

● base ● instructed



SWE-bench

Leaderboards

BENCHMARKS

SWE-bench

SWE-bench Verified

SWE-bench Multilingual

SWE-bench Multimodal

SWE-bench Lite

ABOUT

Paper

Docs

Blog

Contact

Citations

Press

Submit

SWE-BENCH FAMILY

CodeClash

mini-SWE-agent

ProgramBench

SWE-agent (leadcv)

Official Leaderboards

VerifiedMultimodalMultilingualLiteFull

Verified is a human-filtered subset of 500 instances (details). Defaults to *bash-only* setting (run with *mini-SWE-agent*).

Compare results

Bash Only

AGENT mini-SWE-agent

	#	MODEL	AGENT	% RESOLVED	AVG. \$	TRAJS	ORG	DATE	RELEASE
	1	Claude 4.5 Opus (high)	mini-SWE-agent	76.80	\$0.75			2026-02-17	2.0.0
	2	Gemini 3 Flash (high)	mini-SWE-agent	75.80	\$0.36			2026-02-17	2.0.0
	3	MiniMax M2.5 (high)	mini-SWE-agent	75.80	\$0.07			2026-02-17	2.0.0
	4	Claude 4.6 Opus	mini-SWE-agent	75.60	\$0.55			2026-02-17	2.0.0
	5	Claude 4.5 Opus (medium)	mini-SWE-agent	74.40	\$0.72			2025-11-24	1.16.0
	6	Gemini 3 Pro Preview	mini-SWE-agent	74.20	\$0.46			2025-11-18	1.15.0
	7	GLM 5 (high)	mini-SWE-agent	72.80	\$0.53			2026-02-17	2.0.0
	8	GPT 5.2 (high)	mini-SWE-agent	72.80	\$0.47			2026-02-17	2.0.0
	9	GPT 5.2 Codex	mini-SWE-agent	72.80	\$0.45			2026-02-19	2.0.0
	10	GPT 5.2 (high)	mini-SWE-agent	71.80	\$0.52			2025-12-11	1.17.2
	11	Claude 4.5 Sonnet (high)	mini-SWE-agent	71.40	\$0.66			2026-02-17	2.0.0
	12	Kimi K2.5 (high)	mini-SWE-agent	70.80	\$0.15			2026-02-17	2.0.0

Each entry reports % Resolved, the percentage of task instances solved.

Open-weights modelRun performed or directly checked by the SWE-bench team

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

Carlos E. Jimenez^{*1,2} **John Yang**^{*1,2} **Alexander Wettig**^{1,2}

Shunyu Yao^{1,2} **Kexin Pei**³ **Ofir Press**^{1,2} **Karthik Narasimhan**^{1,2}

¹Princeton University ²Princeton Language and Intelligence ³University of Chicago

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

Carlos E. Jimenez^{* 1,2} John Yang^{* 1,2} Alexander Wettig^{1,2}

Shunyu

¹Princeton

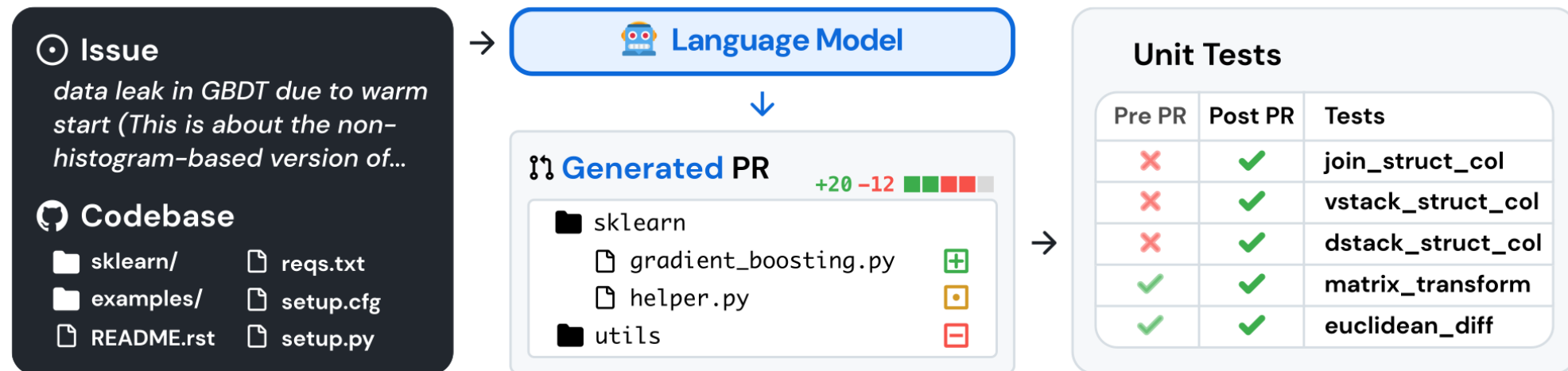


Figure 1: SWE-bench sources task instances from real-world Python repositories by connecting GitHub issues to merged pull request solutions that resolve related tests. Provided with the issue text and a codebase snapshot, models generate a patch that is evaluated against real tests.

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD

Carlos R.

Shunyu

¹Princeton

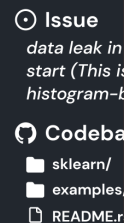


Figure 1: Screenshot of a GitHub issue text and a codebase link.

Model Input

▼ Instructions

• 1 line

You will be provided with a partial code base and an issue statement explaining a problem to resolve.

▼ Issue

• 67 lines

napoleon_use_param should also affect "other parameters" section Subject: napoleon_use_param should also affect "other parameters" section

Problem

Currently, napoleon always renders the Other parameters section as if napoleon_use_param was False, see source

```
def _parse_other_parameters_section(self, section: str) -> List[str]:
    # type: (unicode) -> List[unicode]
    return self._format_fields(_('Other Parameters'))

def _parse_parameters_section(self, section):
    # type: (unicode) -> List[unicode]
    fields = self._consume_fields()
    if self._config.napoleon_use_param: ...
```

▼ Code

• 1431 lines

► README.rst

• 132 lines

► sphinx/ext/napoleon/docstring.py • 1295 lines

► Additional Instructions

• 57 lines

Gold Patch

```
sphinx/ext/napoleon/docstring.py
def _parse_other_parameters_section(self, section: str) -> List[str]:
-     return self._format_fields(_('Other Parameters')), self._consume_fields()
+     if self._config.napoleon_use_param:
+         # Allow to declare multiple parameters at once (ex: x, y: int)
+         fields = self._consume_fields(multiple=True)
+         return self._format_docutils_params(fields)
+     else:
+         fields = self._consume_fields()
+     return self._format_fields(_('Other Parameters'), fields)
```

Generated Patch

```
sphinx/ext/napoleon/docstring.py
def _parse_other_parameters_section(self, section: str) -> List[str]:
-     return self._format_fields(_('Other Parameters')), self._consume_fields()
+     return self._format_docutils_params(self._consume_fields())
```

Generated Patch Test Results

```
PASSED NumpyDocstringTest (test_yield_types)
PASSED TestNumpyDocstring (test_escape_args_and_kwargs 1)
PASSED TestNumpyDocstring (test_escape_args_and_kwargs 2)
PASSED TestNumpyDocstring (test_escape_args_and_kwargs 3)
PASSED TestNumpyDocstring (test_pep526_annotations)
FAILED NumpyDocstringTest (test_parameters_with_class_reference)
FAILED TestNumpyDocstring (test_token_type_invalid)
===== 2 failed, 45 passed, 8 warnings in 5.16s =====
```

Figure 6: We show an example of an formatted task instance, a model prediction, and the testing framework logs. In the patches, red highlights are deletions. Green highlights are additions.

SWE-BENCH: CAN REAL-WORLD GIT

Carlos R

Shunyu

¹Princeto

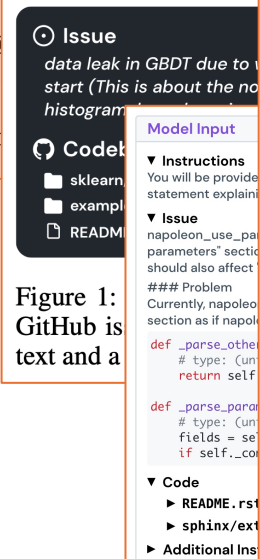


Figure 1: GitHub is text and a

Figure 6: W framework l

4.3 MODELS

Due to the need to process long sequence lengths, there are only a few models that are currently suitable for SWE-bench. Thus we evaluate ChatGPT-3.5 (gpt-3.5-turbo-16k-0613), GPT-4 (gpt-4-32k-0613), Claude 2, and SWE-Llama with their context limits shown in Table 4.

Table 2: Model resolve rates with BM25 retrieval, with different maximum context lengths.

Model	Max. Content		
	13k	27k	50k
Claude 2	1.96	1.87	1.22
SWE-Llama 7b	0.70	0.31	0.00
SWE-Llama 13b	0.70	0.48	0.00

Table 3: BM25 recall with respect to oracle files for different maximum context lengths.

	BM25 Recall		
	13k	27k	50k
Avg.	29.58	44.41	51.06
All	26.09	39.83	45.90
Any	34.77	51.27	58.38

Table 4: We compare the different context lengths and proportion of the “oracle” retrieval setting covered. Models with shorter context lengths are thus inherently disadvantaged. Note that descriptions of token-lengths is a relative non-standard measure (e.g. Llama-tokenized sequences are 42% longer on average than the equivalent sequence tokenized for GPT-4).

	ChatGPT-3.5	GPT-4	Claude 2	SWE-Llama
Max. Tokens	16,385	32,768	100,000	$\geq 100,000$
% of Instances	58.1%	84.1%	96.4%	$\geq 94.8\%$

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE

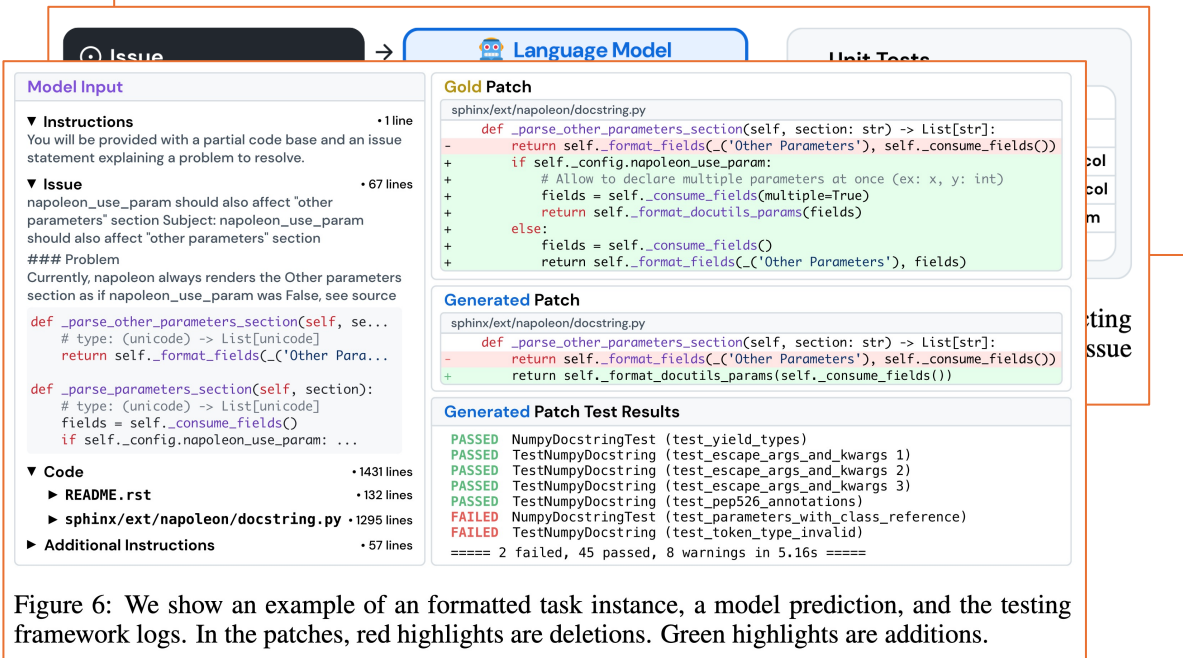


Figure 6: We show an example of an formatted task instance, a model prediction, and the testing framework logs. In the patches, red highlights are deletions. Green highlights are additions.

SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

John Yang* Carlos E. Jimenez* Alexander Wettig Kilian Lieret

Shunyu Yao Karthik Narasimhan Ofir Press

Princeton Language and Intelligence, Princeton University

Table 1: Main results for SWE-agent performance on the full and Lite splits of the SWE-bench test set. We benchmark models in the SWE-agent, Basic CLI, and Retrieval Augmented Generation (RAG) settings established in SWE-bench [20].

Model	SWE-bench		SWE-bench Lite	
	% Resolved	\$ Avg. Cost	% Resolved	\$ Avg. Cost
RAG				
w/ GPT-4 Turbo	1.31	0.13	2.67	0.13
w/ Claude 3 Opus	3.79	0.25	4.33	0.25
Shell-only agent				
w/ GPT-4 Turbo	-	-	11.00	1.46
w/o Demonstration	-	-	7.33	0.79
SWE-agent				
w/ GPT-4 Turbo	12.47	1.59	18.00	1.67
w/ Claude 3 Opus	10.46	2.59	13.00	2.18

August 13, 2024 Milestone

Introducing SWE-bench Verified

We're releasing a human-validated subset of SWE-bench that more reliably evaluates AI models' ability to solve real-world software issues.

[Download SWE-bench Verified ↗](#)

August 13, 2024 Milestone

Introducing SWE-bench Verified

We're releasing a human-validated subset of SWE-bench that more reliably evaluates AI models' ability to solve real-world software issues.

[Download SWE-bench Verified ↗](#)

August 13, 2024 Milestone

Introducing SWE-bench Verified

We're releasing a human-validated subset of SWE-bench that more reliably evaluates AI models' ability to solve real-world software issues.

Our testing identified some SWE-bench tasks which may be hard or impossible to solve, leading to SWE-bench systematically underestimating models' autonomous software engineering capabilities. We've collaborated with the authors of SWE-bench to address those issues in a new release of the benchmark that should provide more accurate evaluations.

August 13, 2024 Milestone

Intro

Our testing identified some SWE-bench tasks which may be hard or impossible to solve, leading to SWE-bench systematically underestimating models' autonomous software

engineer

those iss

evaluation

SWE-bench Verified

To address these issues, we launched a human annotation campaign with professional software developers to screen each sample of the SWE-bench test set for appropriately scoped unit tests and well-specified issue descriptions.

Together with the authors of SWE-bench, we are releasing SWE-bench Verified: a subset of the original test set from SWE-bench, consisting of 500 samples verified to be non-problematic by our human annotators. This version supersedes the original SWE-bench and SWE-bench Lite test sets. Additionally, we are releasing our human annotations for all SWE-bench test samples. These annotations enable slicing the dataset by difficulty. The 'easy' subset is composed of 196 <15-minute fix tasks, while the 'hard' subset is composed of 45 >1-hour tasks.

August 13, 2024 Milestone

Intro

Our testing identified some SWE-bench tasks which may be hard or impossible to solve, leading to SWE-bench systematically underestimating models' autonomous software engineering capabilities.

engineer
those iss
evaluation

SWE-bench Verified

To address
software
scoped u

Together
of the ori
problem
and SWE
all SWE-
The 'easy
compose

Our Approach

We worked with 93 software developers experienced in Python to manually screen SWE-bench samples for quality. We annotated 1,699 random samples from the SWE-bench test set to produce SWE-bench Verified. The following analysis is based on the 1,699 samples.

We annotate samples to capture:

- Whether we consider the issue description to be underspecified and hence unfair to be testing on.
- Whether the `FAIL_TO_PASS` unit tests filter out valid solutions.

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE

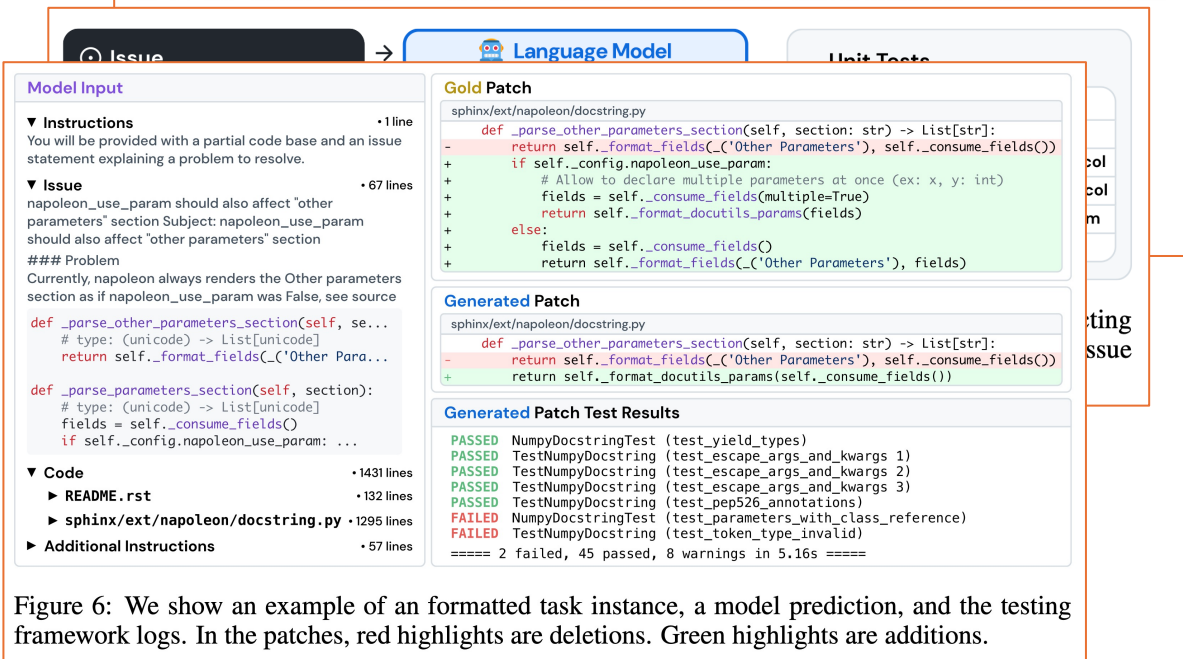


Figure 6: We show an example of an formatted task instance, a model prediction, and the testing framework logs. In the patches, red highlights are deletions. Green highlights are additions.

gpt-oss-120b & gpt-oss-20b Model Card

OpenAI

August 5, 2025

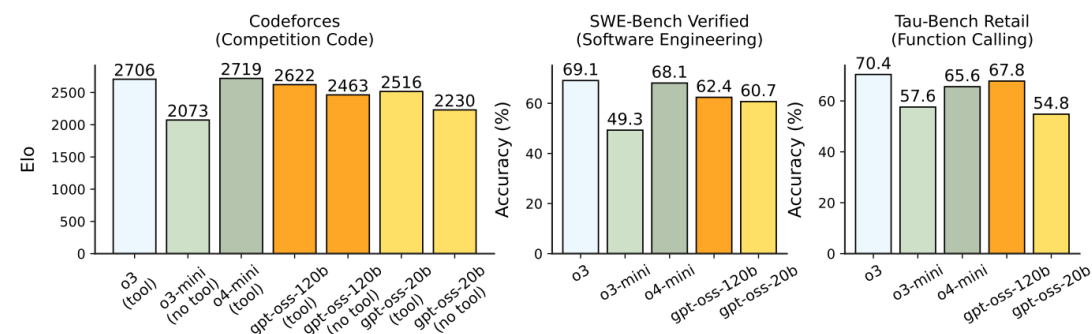


Figure 2: *Coding and tool use results.* To see the models' performance on coding and tool use, we evaluate the gpt-oss models at reasoning level high on a held-out split of Codeforces problems with and without access to a terminal tool. We also evaluate the model on SWE-Bench Verified [22] and evaluate gpt-oss models' developer function using τ -Bench [23]. Similar to the main capability evals, gpt-oss-120b exceeds OpenAI o3-mini, and approaches o4-mini in performance.

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE

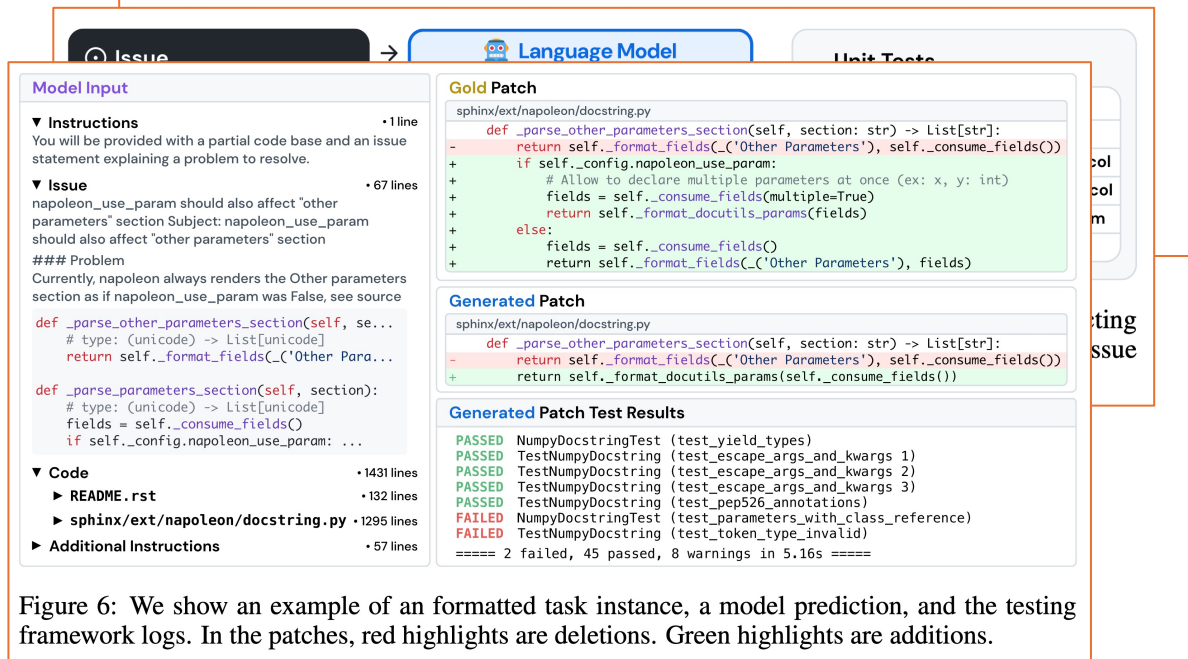
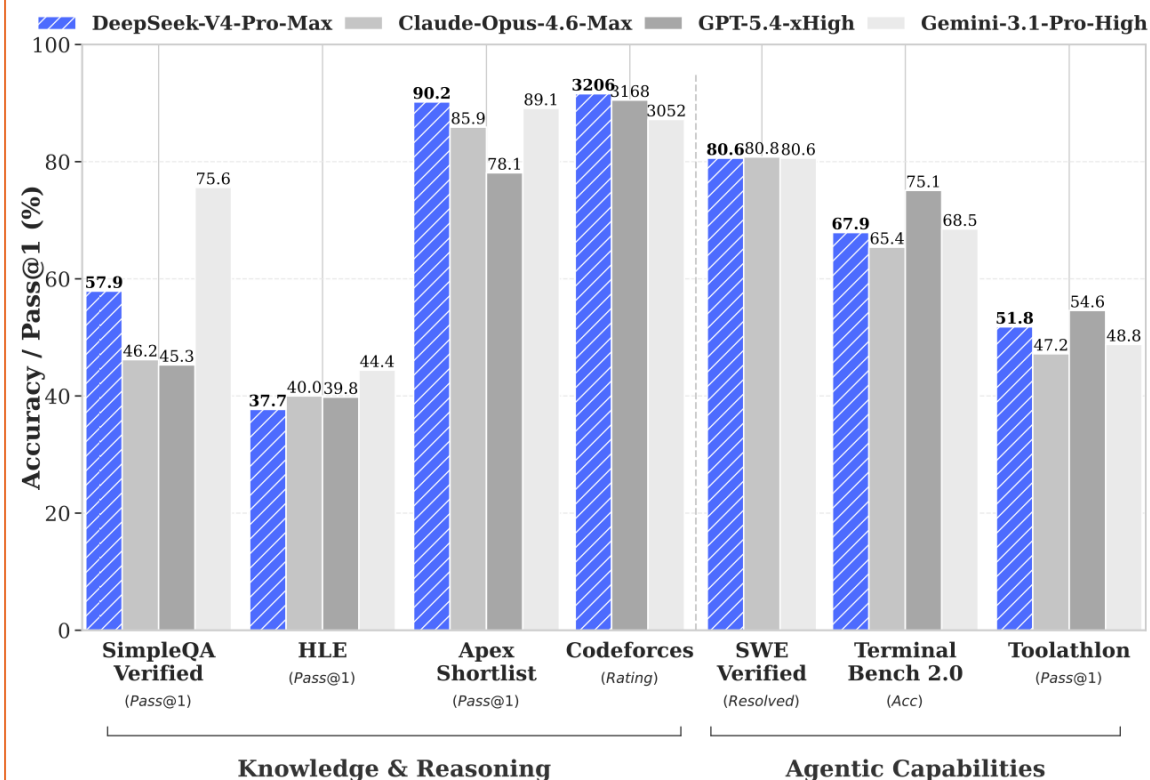


Figure 6: We show an example of an formatted task instance, a model prediction, and the testing framework logs. In the patches, red highlights are deletions. Green highlights are additions.



DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com



February 23, 2026 Research Publication

Why SWE-bench Verified no longer measures frontier coding capabilities

SWE-bench Verified is increasingly
contaminated. We recommend SWE-bench Pro.

Why SWE-bench Verified no longer measures frontier coding capabilities

continued

In a new analysis, we found two major issues with the Verified set that indicate the benchmark is no longer suitable for measuring progress on autonomous software engineering capabilities for frontier launches at today's performance levels:

1. **Tests reject correct solutions:** We audited a 27.6% subset of the dataset that models often failed to solve and found that at least 59.4% of the audited problems have flawed test cases that reject functionally correct submissions, despite our best efforts in improving on this in the initial creation of SWE-bench Verified.
2. **Training on solutions:** Because large frontier models can learn information from their training, it is important that they are never trained on problems and solutions they are evaluated on. This is akin to sharing problems and solutions for an upcoming test with students before the test - they may not memorize the answer but students who have seen the answers before will certainly do better than those without. SWE-bench problems are sourced from open-source repositories many model providers use for training purposes. In our analysis we found that all frontier models we tested were able to reproduce the original, human-written bug fix used as the ground-truth reference, known as the gold patch, or verbatim problem statement specifics for certain tasks, indicating that all of them have seen at least some of the problems and solutions during training.

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?

Xiang Deng*, Jeff Da*

Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson Bing Liu, Brad Kenstler

Scale AI

✉ jeffrey.da@scale.com https://scale.com/research/swe_bench_pro

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon

Xiang Deng*, Jeff Da*
Edwin Pan, Yannis Yimi
Pasari, Chetan Rane, Ka
Zifan Wang, Vijay Bhar
Bing Liu, Brad Kenstler

Scale AI

✉ jeffrey.da@scale.com

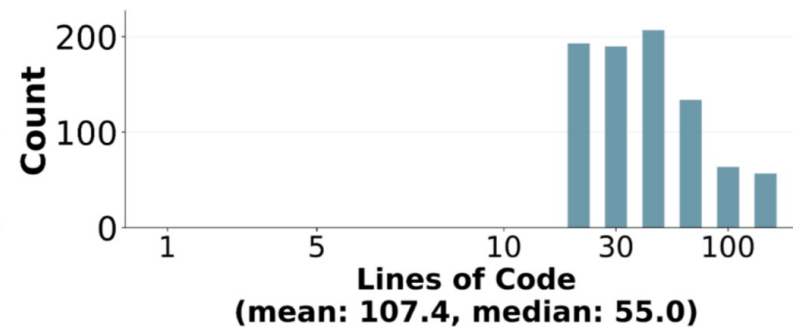
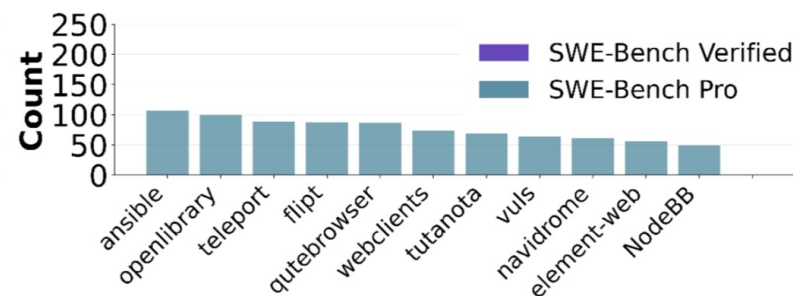
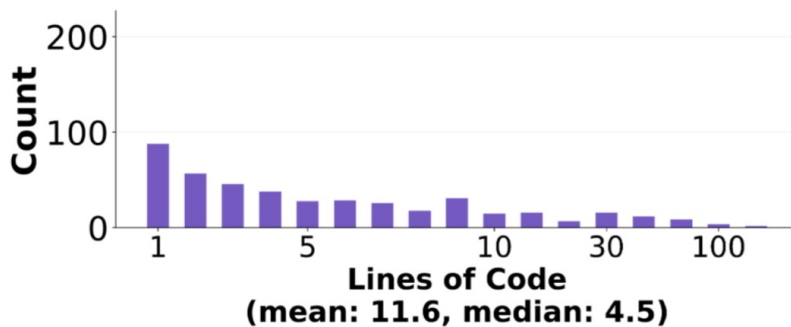
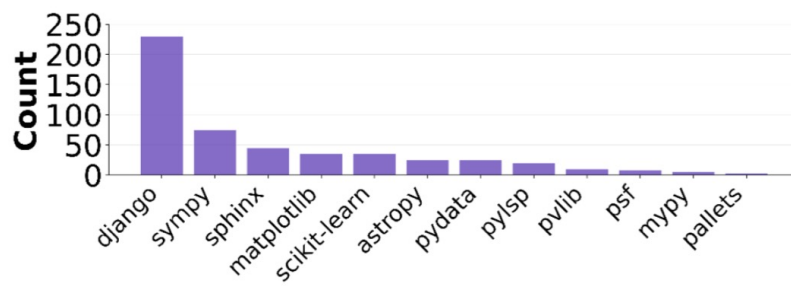


Figure 1: Patches on SWE-BENCH PRO are larger, more challenging, and require expert knowledge about a variety of topics. Patches are generated with SWE-Agent [16] and evaluated on the public subset of SWE-BENCH PRO.

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?

Xiang Deng*, Jeff Da*

Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson Bing Liu, Brad Kenstler

Scale AI

✉ jeffrey.da@scale.com https://scale.com/research/swe_bench_pro

ANTHROPIC

System Card: Claude Opus 5.5

September 22, 2026

anthropic.com

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?

Xiang Deng*, Jeff Da*
Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson Bing Liu, Brad Kenstler

Scale AI
✉ jeffrey.da@scale.com https://scale.com/research/swe_bench_pro

ANTHROPIC					
8 Capabilities					
8.1 Evaluation summary					
Evaluation	Claude family models			Other models	
	Claude Opus 5.5	Claude Opus 5	Claude Fable 5.1	GPT-6 Astra	
SWE-bench Pro	89.9	79.2	81.2	-	
SWE-bench Multilingual	93.9	89.5	89.1	-	
SWE-bench Multimodal	61.4	59.4	54.7	-	
FrontierCode v1.1 (Main)	54.4	48.0	50.3	53.3	
Terminal-Bench 4.0	66.4	52.3	55.8	57.9	
Terminal-Bench-Science 0.1	58.7	29.0	52.6	64.6	
Humanity's Last Exam	No tools	64.4	56.6	60.9	-
	With tools	67.7	63.6	65.6	57.2
OSWorld 2.0 (partial/strict)	81.8/48.7	74.0/37.2	80.7/42.8	-	
HealthBench Professional	65.6	59.8	62.1	63.4	
GDPval-AA v2.1	1846	1708	1735	1542	
AA-Briefcase v1.1	1822	1673	1678	1569	
AutomationBench	40.0	26.9	31.4	41.4	
[Table 8.1.A] Capability evaluation summary. Unless otherwise noted, all Claude Opus 5.5 results use the following standard configuration: adaptive thinking at max effort, default sampling settings (temperature, top_p), averaged over five trials. Terminal-Bench 4.0 score is reported at xhigh effort. Context window sizes are evaluation dependent and do not exceed 1M tokens. The best score in each row is bolded. Competitor figures are drawn from the respective developers' published system cards or benchmark leaderboards. See the Claude Opus 5 System Card for evaluation details of earlier Claude models.					

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?

Xiang Deng*, Jeff Da*
Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson Bing Liu, Brad Kenstler

Scale AI
✉ jeffrey.da@scale.com 🌐 https://scale.com/research/swe_bench_pro

ANTHROPIC

8 Capabilities

8.1 Evaluat

Evaluat

SWE-b

SWE-b

SWE-b

Frontie

Termin

Termin

Human

OSWor

Health

GDPval

AA-Brie

Automa

Table 8.1 following (top_p), av are evalua figures are Claude Op

Hugging Face

Search models, datasets, users...

Models

Datasets

Qwen/Qwen3.8-27B

Like 17.2k

Follow

Qwen 109k

Image-Text-to-Text

Transformers

Safetensors

qwen3_5

conversational

Eval Results

License: apac

Model card

Files and versions

xet

Community 206

Qwen3.8-27B

This repository contains model weights and configuration files for the post-trained model in the Hugging Face Transformers format.

These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang, TokenSpeed, etc.

SWE-Bench Pro: Can AI Agents Solve Long-Horizon SE Tasks?, Deng et al., Scale AI 2025

System Card: Claude Opus 5.5, Anthropic, September 2026

scale

SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?

Xiang Deng*, Jeff Da*
Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson Bing Liu, Brad Kenstler

Scale AI
jeffrey.da@scale.com https://scale.com/research/swe_bench_pro

ANTHROPIC

8 Capabilities

8.1 Eval

Evaluate

 Hugging Face

 Models

 Datasets

SWE-bench

SWE-bench

SWE-bench

Frontier

Terminus

Terminus

Human

OSWorld

HealthBench

GDPValue

AA-Bridge

AutomationBench

Qwen3.8-27B

Qwen3.6-27B

Qwen3.7-Plus

Muse Glimmer-30B

Opus4.6 Max

Coding

Agentic terminal coding

Terminal Bench 2.1 (Terminus)

73.0

63.4

64.0

51.7

78.2

Agentic coding

SWE-bench Pro

61.7

53.5

57.6

51.2

53.4

Repo-level code generation

NL2Repo-Bench

42.3

36.2

41.1

--

47.6

Agentic coding

DeepSWE 1.1

42.2

13.3

14.2

--

--

Software engineering

QwenSWEBench

79.0

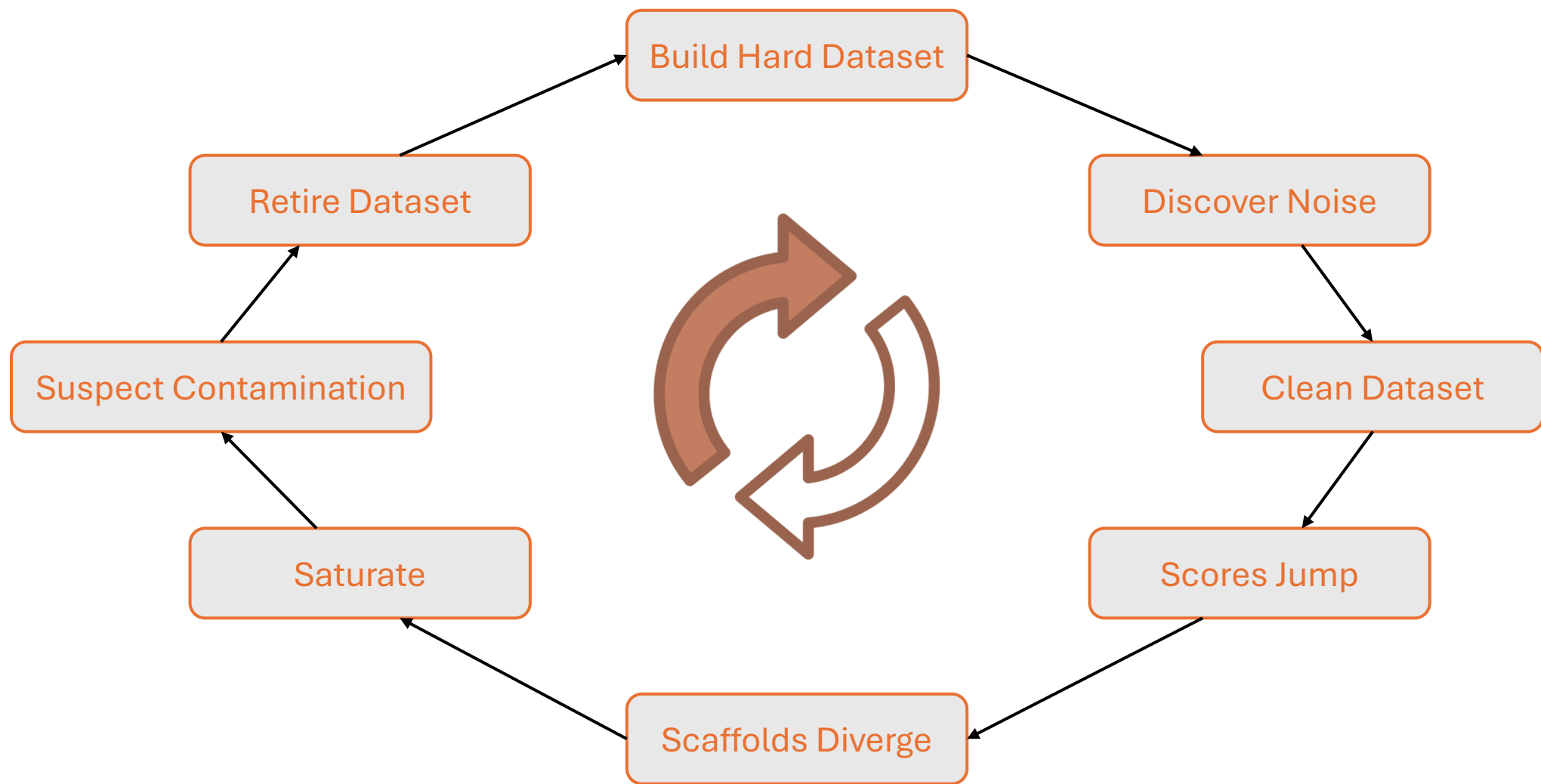
49.3

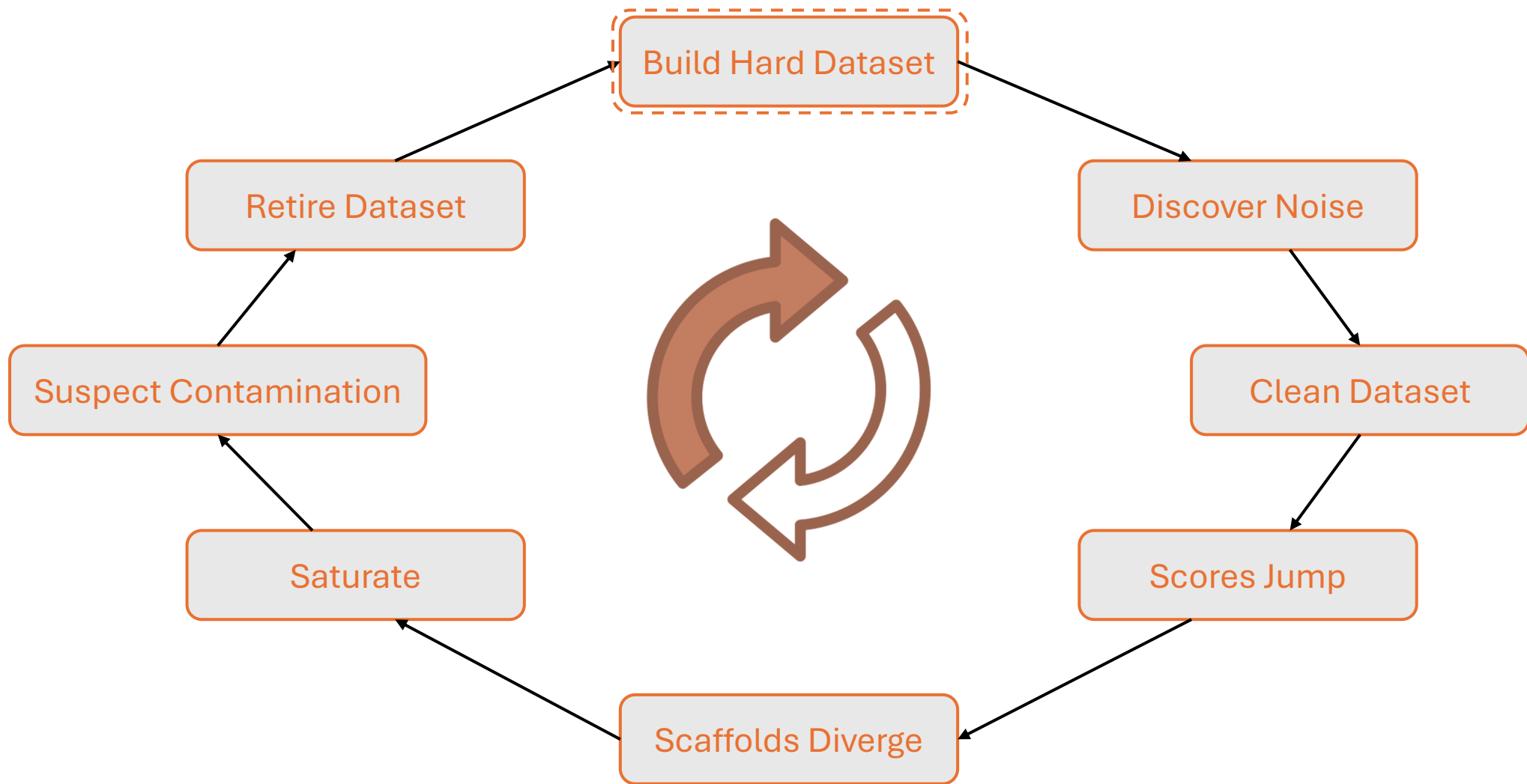
59.2

--

63.8

Table 8.1A] Capabilities following standard configuration (top_p), averaged over 100 evaluation dependent figures are drawn from Claude Opus 5.5 System





Vero: Can AI Agents Build Formally Verified Software Repositories?

Zhe Ye^{1*} Hantao Lou^{1*} Yuechun Sun^{2*} Peiyang Song³ Zhengxu Yan⁴ Timothe Kasriel¹

Qingyang Zhang¹ Kaiyu Yang⁵ Soonho Kong^{6†} Jingxuan He¹ Dawn Song¹

¹UC Berkeley ²University of Chicago ³California Institute of Technology

⁴Stanford University ⁵Apodex ⁶Amazon Web Services

Vero: Can AI

3 The Vero Benchmark

Zhe Ye^{1*} Hantao Lou¹

Qingyang Zhang¹

¹UC Berkeley

⁴Stanford

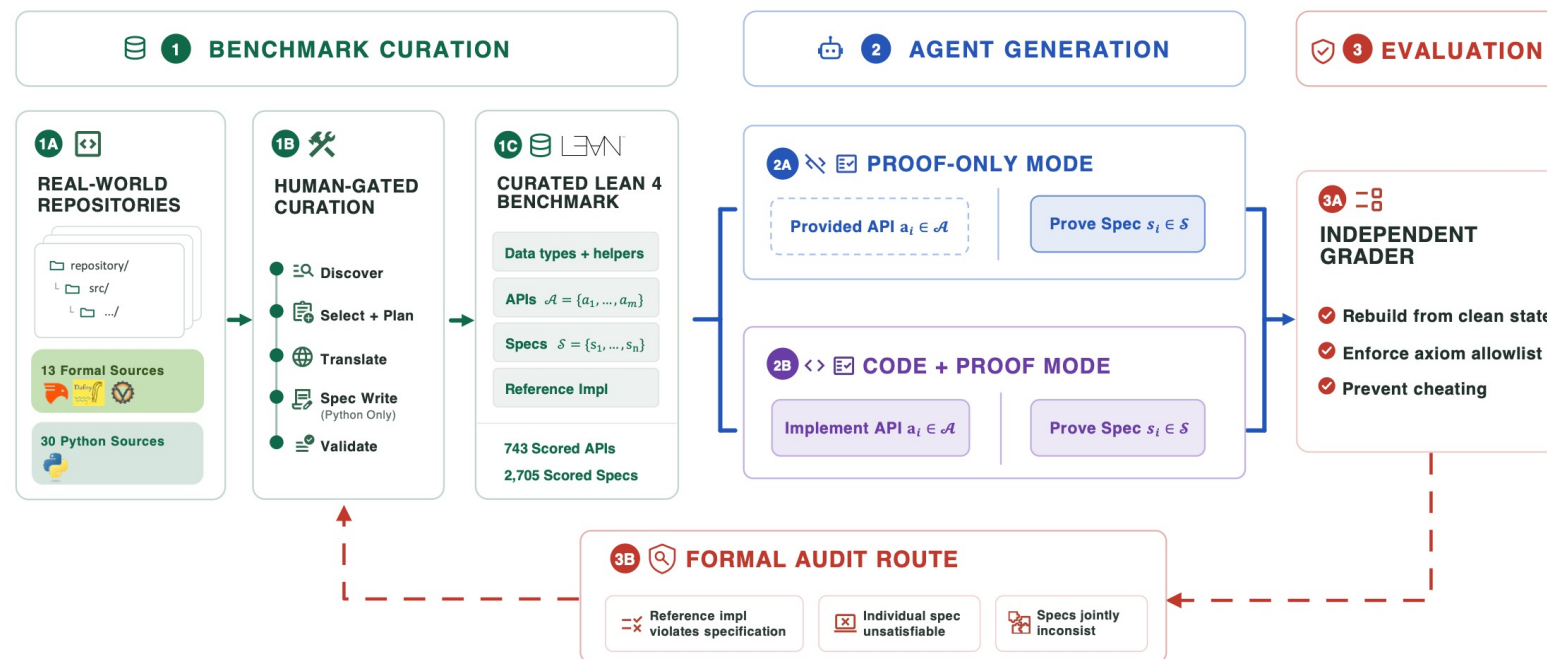


Figure 1: **Vero’s end-to-end construction and evaluation workflow.** Human-gated curation converts real-world Python and formal-language repositories into Lean 4 benchmark instances with fixed definitions, API signatures, specifications, and reference implementations. Agents are evaluated in proof-only or code-and-proof mode and scored by an independent grader, while the formal audit route returns machine-checked evidence of benchmark defects to curation.



Figure 1: Vero’s interface showing real-world Python repositories, API signatures, and the option to run in proof-only or code-and-proof mode.

```

1  -- API signatures
2  abbrev CreateAccountSig := AccountId -> Ledger -> Ledger
3  abbrev AccountExistsSig := AccountId -> Ledger -> Bool
4  abbrev GetBalanceSig    := AccountId -> Ledger -> Option Balance
5
6  -- Interface structure: one field per API signature
7  structure RepoImpl where
8    createAccount : CreateAccountSig
9    accountExists : AccountExistsSig
10   getBalance    : GetBalanceSig
11
12  -- Canonical implementation (instantiated to reference implementation in proof-
13   only mode and agent solution in code-and-proof mode)
14  def canonical : RepoImpl := {
15    createAccount := Bank.createAccount,
16    accountExists := Bank.accountExists,
17    getBalance    := Bank.getBalance,
18  }
19
20  -- Specification: a predicate over implementations
21  def spec_create_zero_balance (impl : RepoImpl) : Prop :=
22    ∀ (id : AccountId) (ledger : Ledger),
23      impl.accountExists id ledger = false ->
24      impl.getBalance id (impl.createAccount id ledger) = some 0
25
26  -- Theorem: Proof obligation on the canonical implementation
27  theorem proof_create_zero_balance : spec_create_zero_balance canonical := by
28    sorry

```

Figure 2: Excerpt from a Vero example illustrating a benchmark instance. Frozen content like API signatures and specifications are provided by the curator. The agent fills in the body of `canonical` (in code-and-proof mode) and the proof obligation marked `sorry` (in both mode).

3 The Vero Benchmark

```

1  -- API signatures
2  abbrev CreateAccountSig := AccountId -> Ledger -> Ledger
3  abbrev
4  abbrev
5
6  --
7  str
8  c:
9  a
10 g
11
12 --
13 def
14 c:
15 a
16 g
17 }
18
19 --
20 def
21
22
23
24
25 --
26 the

```

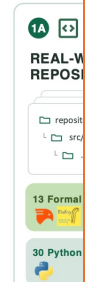


Figure
real-w
defini
proof-
route

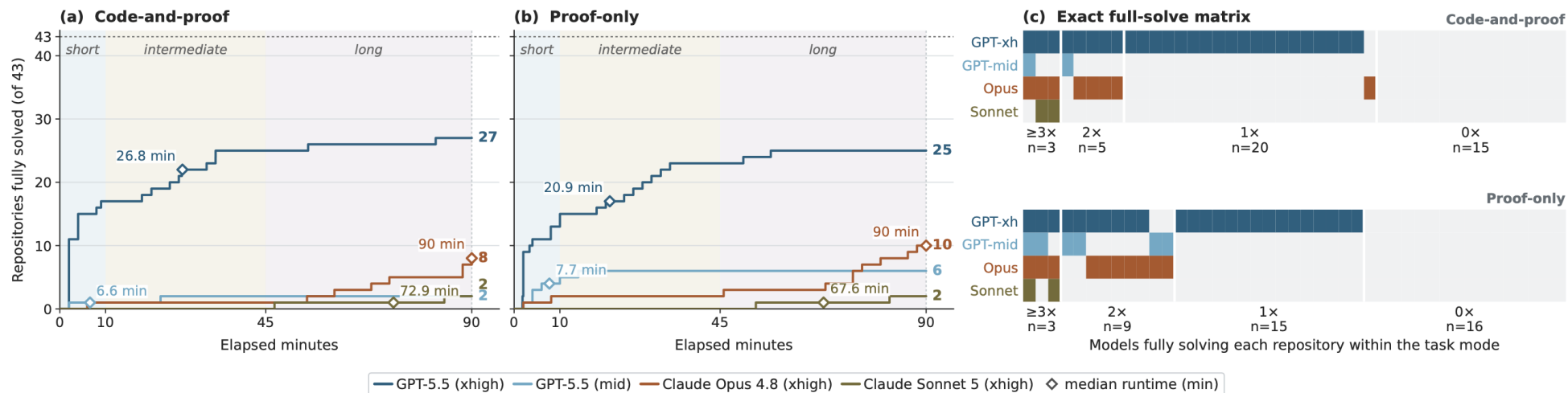
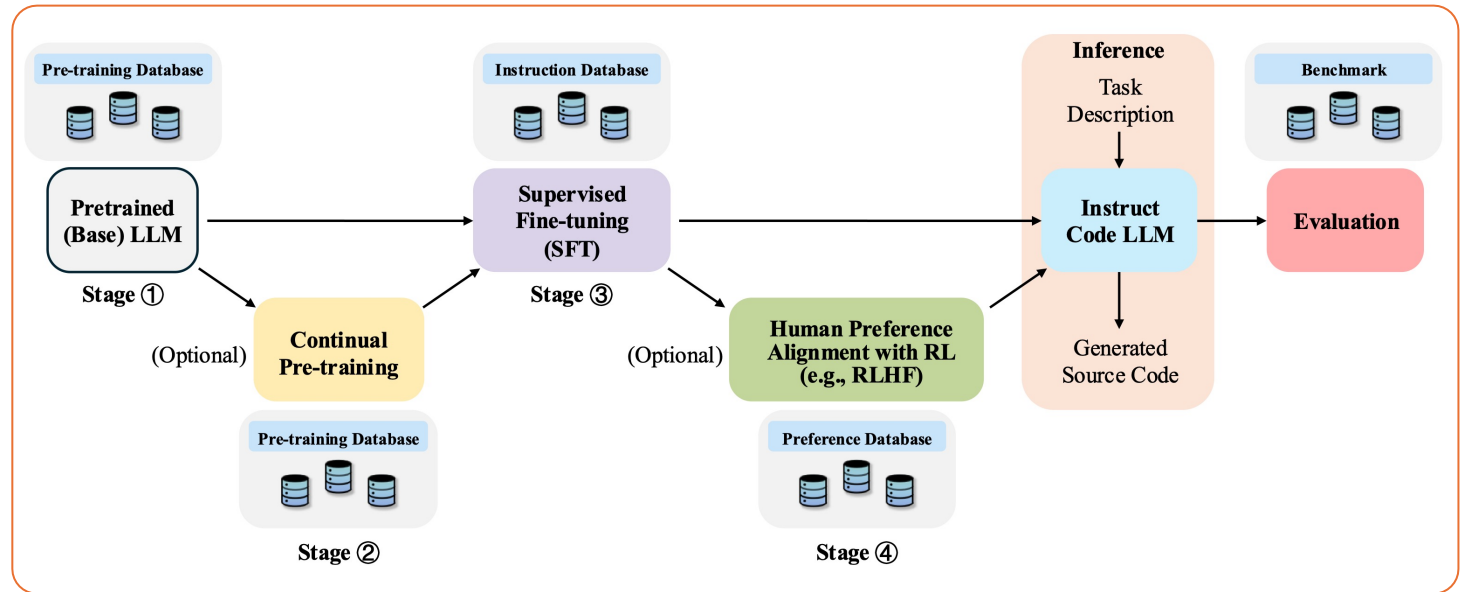


Figure 3: Agent performance on Vero. (a,b) Cumulative full solves, out of 43, over the 90-minute budget in code-and-proof and proof-only; diamonds mark each configuration’s median runtime. (c) Which configurations fully solve each instance in each mode.

Figure 2: Excerpt from a Vero example illustrating a benchmark instance. Frozen content like API signatures and specifications are provided by the curator. The agent fills in the body of canonical (in code-and-proof mode) and the proof obligation marked sorry (in both mode).

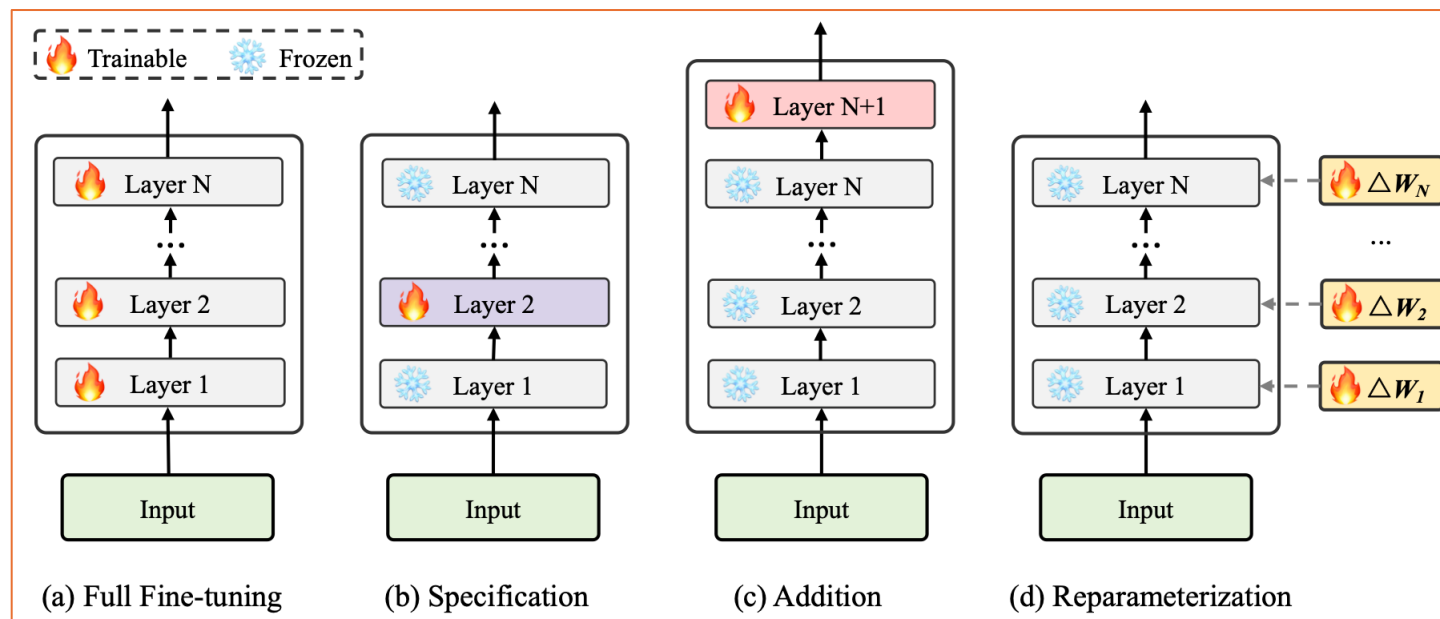
Today's Agenda

- Pre-training stage
 - ~~Model architecture~~
 - ~~Pre-training dataset~~
 - ~~Learning objectives~~
 - ~~Evaluation dataset~~
- Special topics
 - Post-training staging
 - Scaling law
 - Hallucination



Post-Training Staging

- Instruction tuning
- Full parameter fine-tuning (FFT)
- Supervised fine-tuning (SFT)
- Parameter-efficient fine-tuning (PEFT)
- Reinforcement learning (RL)
 - Human Feedback
 - Logical Feedback
 - Compiler Feedback
 - LLM-as-judge Feedback



Scaling Law

- “Compute”: FLOP
 - FLOP: Floating point operations
 - Total training compute that aggregates over model size, dataset size and training duration
- Approximation:
 - $\text{FLOPs} \approx 6 \times N \times D$
 - N : number of model parameters
 - D : number of dataset tokens
 - Factor 6: forward + backward passes + architecture constants

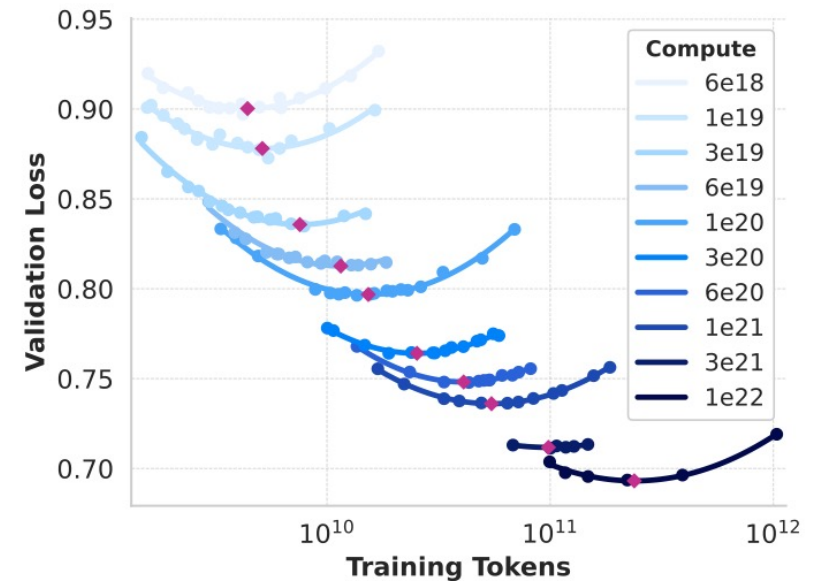


Figure 2 Scaling law IsoFLOPs curves between 6×10^{18} and 10^{22} FLOPs. The loss is the negative log-likelihood on a held-out validation set. We approximate measurements at each compute scale using a second degree polynomial.

Scaling Law

Observational Scaling Laws and the Predictability of Language Model Performance

Yangjun Ruan^{1,2,3}
`yjruan@cs.toronto.edu`

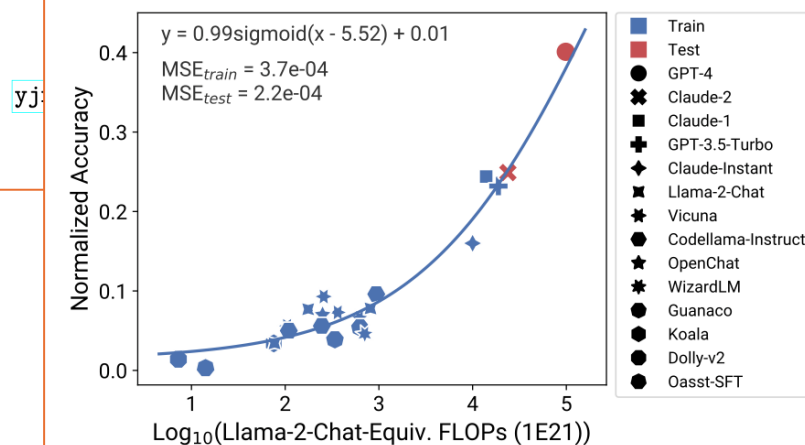
Chris J. Maddison^{2,3}
`cmaddis@cs.toronto.edu`

Tatsunori Hashimoto¹
`thashim@stanford.edu`

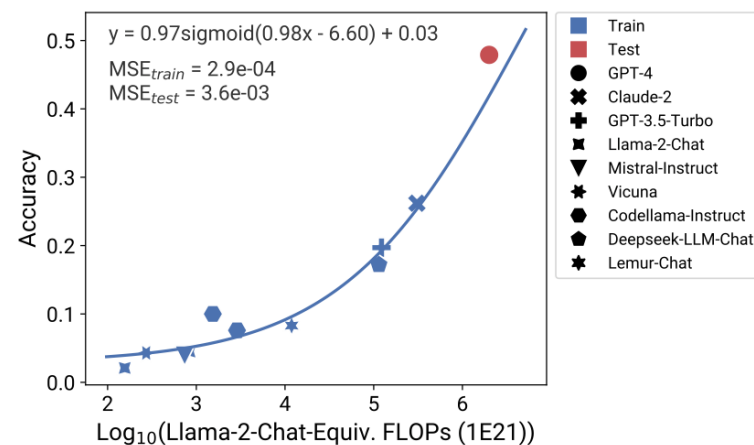
¹Stanford University ²University of Toronto ³Vector Institute

Scaling Law

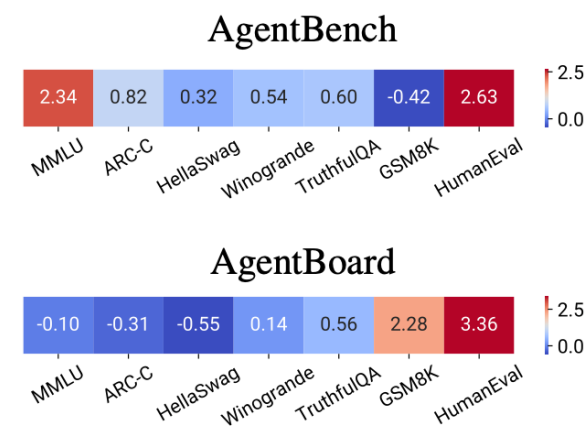
Observational Scaling Laws and the Predictability of Language Model Performance



(a) AgentBench



(b) AgentBoard



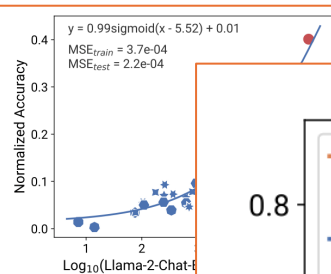
(c) Weight visualization

Scaling Law

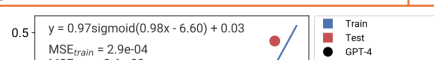
Observational Scaling Laws and

the I

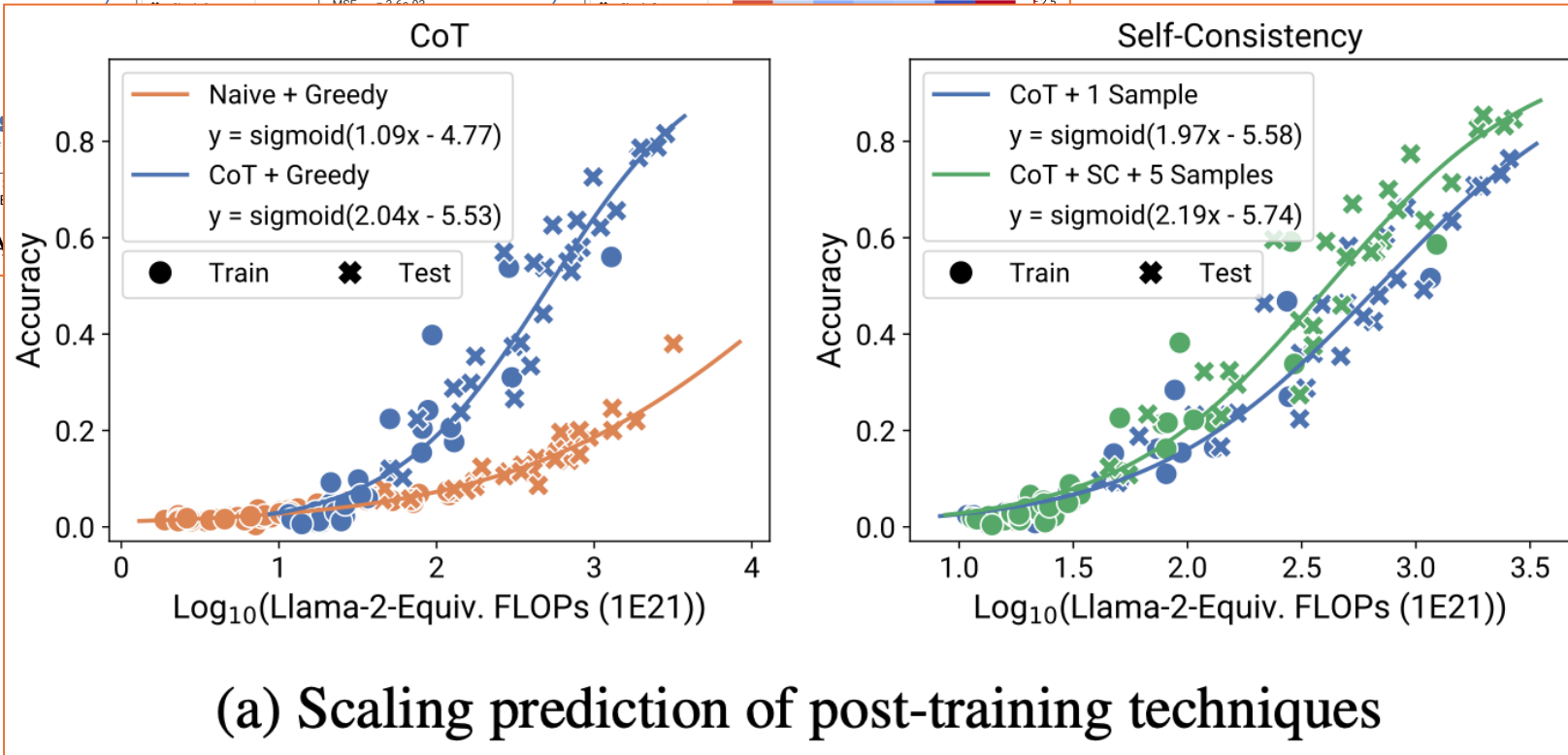
Yar
yjrur



(a) A



AgentBench



(a) Scaling prediction of post-training techniques

Hallucination

A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions

LEI HUANG, Harbin Institute of Technology, China

WEIJIANG YU, Huawei Inc., China

WEITAO MA and WEIHONG ZHONG, Harbin Institute of Technology, China

ZHANGYIN FENG and HAOTIAN WANG, Harbin Institute of Technology, China

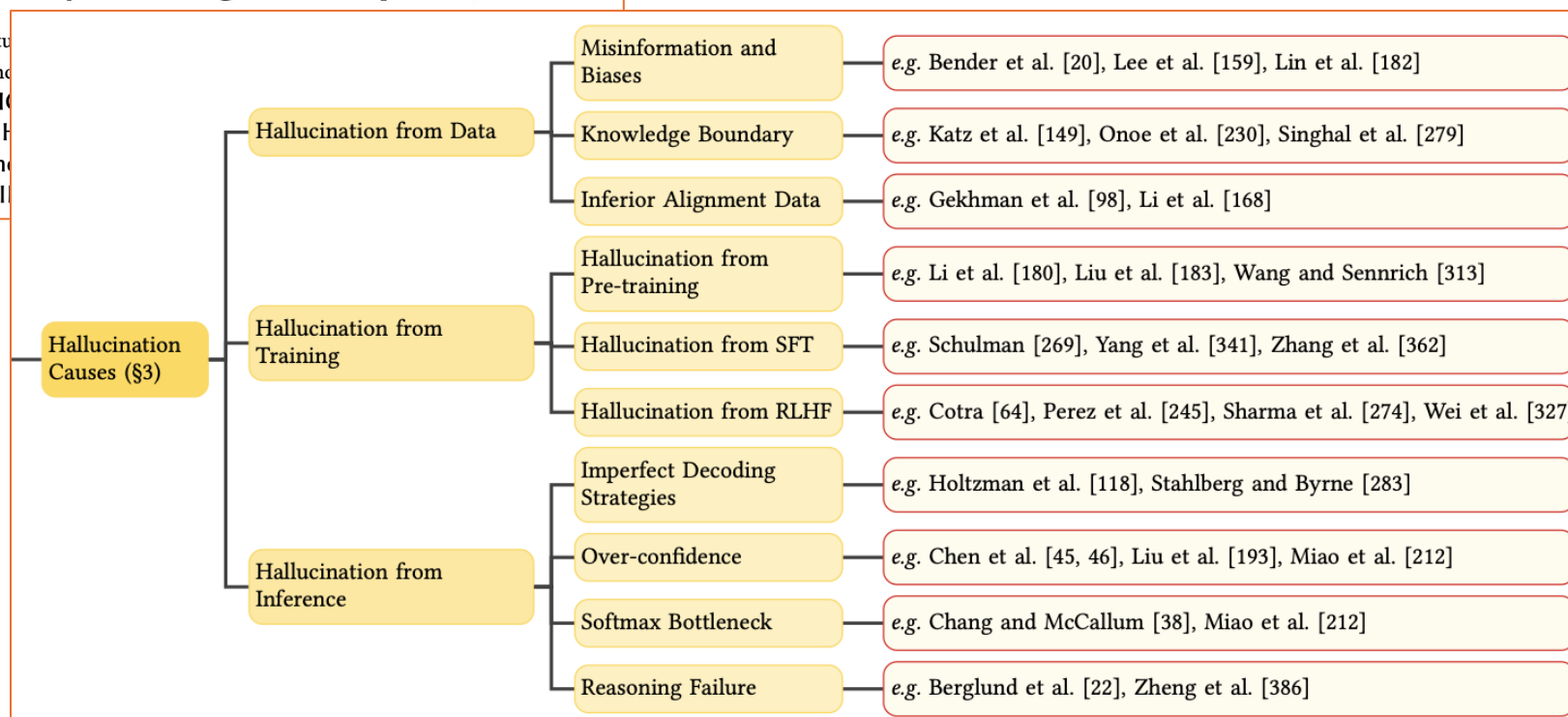
QIANGLONG CHEN and WEIHUA PENG, Huawei Inc., China

XIAOCHENG FENG*, BING QIN, and TING LIU, Harbin Institute of Technology, China

Hallucination

A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions

LEI HUANG, Harbin Institute of Technology
WEIJIANG YU, Huawei Inc.
WEITAO MA and WEIHONG ZHANG
ZHANGYIN FENG and HONG QIANG
QIANGLONG CHEN and XIAOCHENG FENG*, Baidu



Hallucination

A Survey on Hallucination Principles

LEI HUA
WEIJIAN
WEITAO
ZHANG
QIANG
XIAOCH

Hallucination Causes (§3)

Hallucination

Hallucination Training

Hallucination Inference

Table 1. Examples of each category of LLM hallucinations. Content marked in **Red** represents the hallucinatory output, while content marked in **Blue** indicates user instruction or provided context that contradicts the LLM hallucination.

Category	Type	Example Dialog	Explanation
Factuality Hallucination	Factual Contradiction	Instruction: What are Thomas Edison’s main contributions to science and technology? Response: Thomas Edison developed the first practical telephone and invented the light bulb.	The response is factually incorrect. In reality, Edison improved the light bulb, building on earlier designs by others , and Alexander Graham Bell invented the telephone .
	Factual Fabrication	Instruction: What are the major environmental impacts of the construction of the Eiffel Tower? Response: The construction of the Eiffel Tower in 1889 led to the extinction of the Parisian tiger , a species that played a crucial role in the region’s ecosystem. Additionally, it is widely recognized as the event that sparked the global green architecture movement .	The response erroneously states that the construction of the Eiffel Tower led to the extinction of the ‘Parisian tiger’ and sparked the global green architecture movement. In fact, there is no evidence of a ‘Parisian tiger’ ever existing, making this a fabricated claim . Moreover, attributing the origins of green architecture to the Eiffel Tower is an exaggeration , as this movement has diverse roots and cannot be traced back to a single event.
	Instruction Inconsistency	Instruction: Translate the English question into Spanish: “What is the capital of France?” Response: The capital of France is Paris.	The user explicitly provided an instruction to translate the English question into Spanish. Instead of performing the translation as instructed, the LLM chose to answer the question.
Faithfulness Hallucination	Context Inconsistency	Retrieved context: The Nile is a major north-flowing river in northeastern Africa. It’s traditionally considered the longest river in the world. Originating in the Great Lakes region of central Africa , it flows through several countries before emptying into the Mediterranean Sea. Instruction: Summarize the retrieval document above. Response: The Nile originates from the mountain ranges of central Africa and flows into the Mediterranean.	The summary incorrectly states that the Nile originates from the “mountain ranges of central Africa”, while the paragraph mentions that the Nile originates in the Great Lakes region of central Africa .
	Logical Inconsistency	Instruction: Solve the equation $2x + 3 = 11$ step by step. Response: Step 1: Subtract 3 from both sides to get: $2x = 8$. Step 2: Divide both sides by 2 to get: $x = 3$. So the result of the equation is $x=3$.	The first step correctly isolates the term with x to get $2x = 8$. However, the next step inaccurately divides 8 by 2 to yield a result of $x = 3$, which is inconsistent with the earlier reasoning.

Hallucination

Exploring and Evaluating Hallucinations in LLM-Powered Code Generation

Fang Liu*, Yang Liu*, Lin Shi[†], Houkun Huang*, Ruifeng Wang*, Zhen Yang[‡], Li Zhang*
Zhongqi Li[§], Yuchi Ma[§]

*School of Computer Science and Engineering, Beihang University, Beijing, China

[†]School of Software, Beihang University, Beijing, China

[‡]School of Computer Science and Technology, Shandong University, Qingdao, China

[§]Huawei Cloud Computing Technologies Co., Ltd, China

{fangliu, liuyang26, shilin, huanghoukun, ruifengwang}@buaa.edu.cn, zhenyang@sdu.edu.cn, lily@buaa.edu.cn
{lizhongqi7, mayuchi1}@huawei.com

Hallucination

Exploring and Evaluating Hallucinations in LLM-Powered Code Generation

Fang Liu*, Yang Liu*, L

*School of Compu

†Sch

‡School of Comput

§Huav

{fangliu, liuyang26, shilin, huang

Code Hallucination Taxonomy

Intent Conflicting

overall semantic conflicting

local semantic conflicting

Context Deviation

Inconsistency

expression

constant

loop/condition/branch

loop

Repetition

copy input context

generate repetitive statements

Dead Code

IO/assert statements

loop/condition/branch

function definition

assignment

Knowledge Conflicting

API knowledge

using un-imported library

using wrong/extra library

missing library

using wrong/extra parameters

miss parameters

Identifier knowledge

using undefined identifiers

using wrong identifiers

```
def string_xor(a: str, b: str) -> str:
    """ Input are two strings a and b
    consisting only of 1s and 0s.
    Perform binary XOR on these inputs
    and return result also as a string.
    >>> string_xor('010', '110')
    '100'
    """
```

generated by CodeRL

```
a, b = a.strip('0'), b.strip('0')
res = ''
if a != b:
    res = '0' + res
if a:
    res = '1' + res
if b:
    res = '0' + res
return res
```

Conflicting

reference solution

```
def xor(i, j):
    if i == j:
        return '0'
    else:
        return '1'
return ''.join(xor(x, y) for x, y
                in zip(a, b))
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
x = np.arange(10)
y = np.sin(x)
df = pd.DataFrame({"x": x, "y": y})
sns.lineplot(x="x", y="y", data=df)
```

generated by CodeGen

```
df = pd.DataFrame({"x": x, "y": y})
sns.lineplot(x="x", y="y", data=df)
```

reference solution

```
ax = plt.gca()
ax.set(xlabel=None)
```

Repea

Example 5. Repetition: copy input.

```
def even_odd_count(num):
    """Given an integer. return a tuple
    that has the number of even and
    odd digits respectively.
    Example:
    even_odd_count(-12) ==> (1, 1)
    even_odd_count(123) ==> (1, 2)
    """
```

```
even_count = 0
odd_count = 0
```

generated by GPT-3.5

```
num_str = str(abs(num))
for digit in num_str:
    if int(digit) % 2 == 0:
        even_count += 1
    else:
        odd_count += 1
if num < 0:
    even_count *= -1
    odd_count *= -1
return (even_count, odd_count)
```

reference solution

```
for i in str(abs(num)):
    if int(i)%2==0:
        even_count +=1
    else:
        odd_count +=1
return (even_count, odd_count)
```

Example 2. Intent conflicting: local semantic conflict

```
"""
Problem:
I have an array :
a = np.array([[ 0,  1,  2,  3,  5,  6,  7,  8],
               [ 4,  5,  6,  7,  5,  3,  2,  5],
               [ 8,  9, 10, 11,  4,  5,  3,  5]])
I want to extract array by its columns in RANGE, if I want
to take column in range 1 until 10, It will return
a = np.array([[ 1,  2,  3,  5,  6,  7,  8],
               [ 5,  6,  7,  5,  3,  2,  5],
               [ 9, 10, 11,  4,  5,  3,  5]])
Pay attention that if the high index is out-of-bound, we
should constrain it to the bound.
How to solve it? Thanks
"""
```

```
import numpy as np
a = np.array([[ 0,  1,  2,  3,  5,  6,  7,  8],
               [ 4,  5,  6,  7,  5,  3,  2,  5],
               [ 8,  9, 10, 11,  4,  5,  3,  5]])
```

```
low = 1
high = 10
```

Inconsistent

generated by GPT-3.5

```
result = a[:, max(0, low-1):min(high, a.shape[1])]
```

reference solution

```
high = min(high, a.shape[1])
result = a[:, low:high]
```

Example 3. Context inconsistency: inaccurate expression

Conflicting

```
def filter_by_substring(strings: List[str], substring: str)
-> List[str]:
```

```
""" Filter an input list of strings only for
ones that contain given substring
```

```
>>> filter_by_substring([], 'a')
[]
>>> filter_by_substring(['abc', 'bacd', 'cde',
                          'array'], 'a')
['abc', 'bacd', 'array']
"""
```

Inconsistent

generated by CodeRL

```
return [string for string in strings if
        string.startswith(substring)]
```

reference solution

```
return [x for x in strings if substring in x]
```

Example 4. Context inconsistency: inaccurate condition.

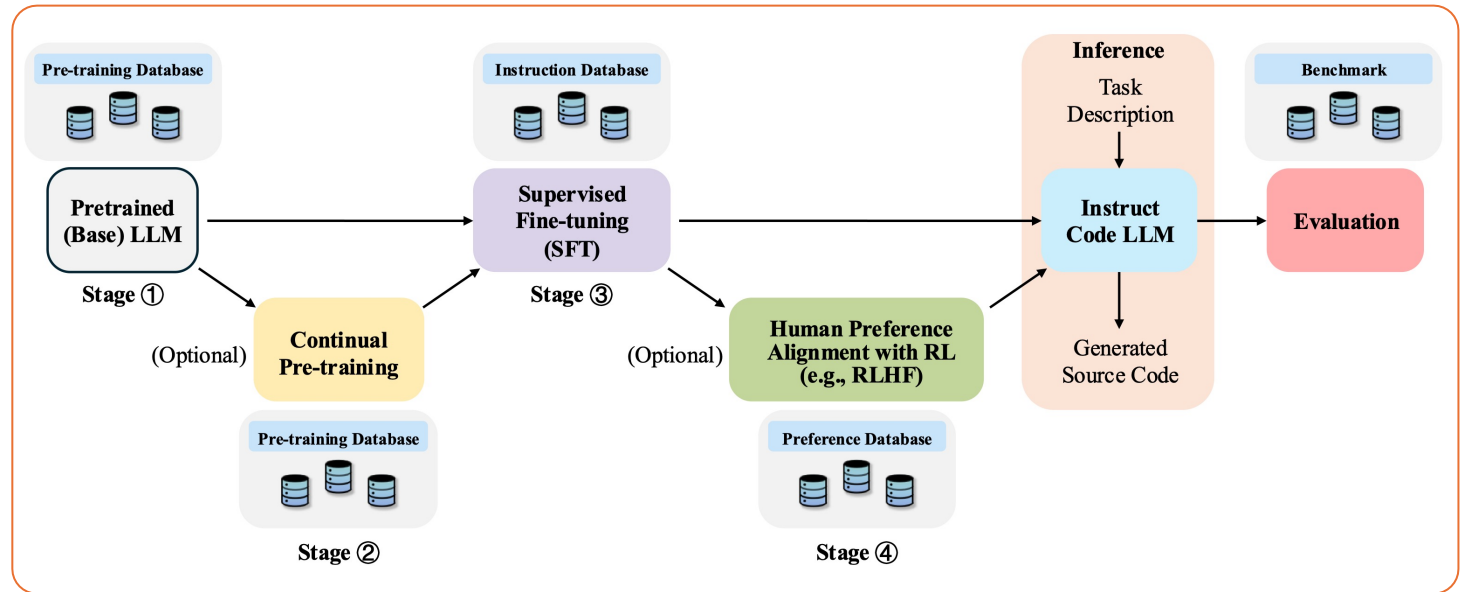
Today's Agenda

- Pre-training stage

- ~~Model architecture~~
- ~~Pre-training dataset~~
- ~~Learning objectives~~
- ~~Evaluation dataset~~

- Special topics

- ~~Post-training staging~~
- ~~Scaling law~~
- ~~Hallucination~~



How to obtain a “good enough” LLM

